

대분류/19
전기·전자

중분류/03
전자기기개발

소분류/08
로봇개발

세분류/03
로봇소프트웨어개발

능력단위/10

NCS학습모듈

로봇 소프트웨어 시험평가

LM1903080310_14v1



교육부

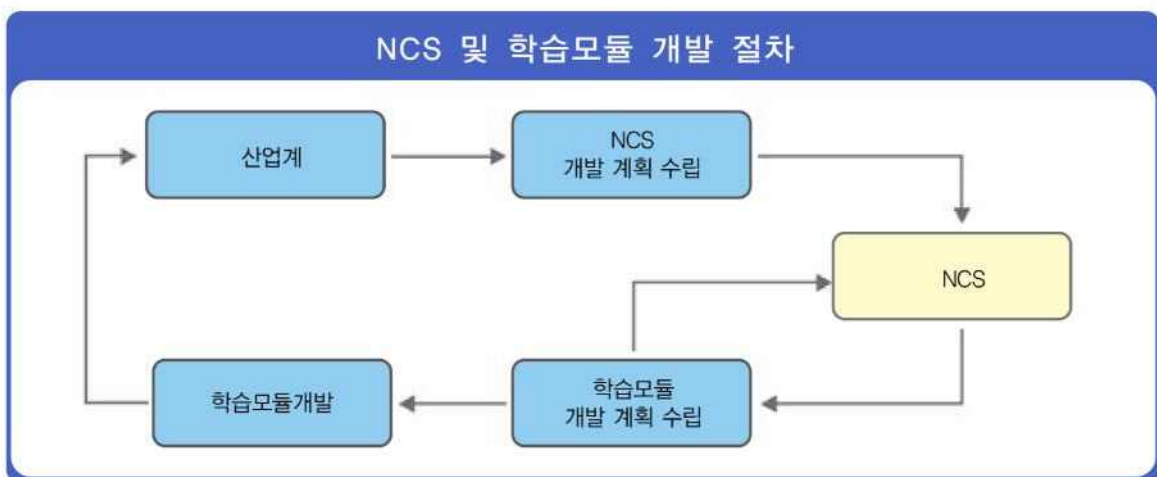
NCS 학습모듈은 교육훈련기관에서 출처를 명시하고 교육적 목적으로 활용할 수 있습니다. 다만 NCS 학습모듈에는 국가(교육부)가 저작권 일체를 보유하지 않은 저작물들(출처가 표기되어 있는 도표, 사진, 삽화, 도면 등)이 포함되어 있으므로 이러한 저작물들의 변형, 복제, 공연, 배포, 공중 송신 등과 이러한 저작물들을 활용한 2차 저작물의 생성을 위해서는 반드시 원작자의 동의를 받아야 합니다.

NCS 학습모듈의 이해

※ 본 학습모듈은 「NCS 국가직무능력표준」 사이트(<http://www.ncs.go.kr>) 에서 확인 및 다운로드 할 수 있습니다.

(1) NCS 학습모듈이란?

- 국가직무능력표준(NCS: National Competency Standards)이란 산업현장에서 직무를 수행하기 위해 요구되는 지식·기술·소양 등의 내용을 국가가 산업부문별·수준별로 체계화한 것으로 산업현장의 직무를 성공적으로 수행하기 위해 필요한 능력(지식, 기술, 태도)을 국가적 차원에서 표준화한 것을 의미합니다.
- 국가직무능력표준(이하 NCS)이 현장의 ‘직무 요구서’라고 한다면, NCS 학습모듈은 NCS의 능력단위를 교육훈련에서 학습할 수 있도록 구성한 ‘교수·학습 자료’입니다. NCS 학습모듈은 구체적인 직무를 학습할 수 있도록 이론 및 실습과 관련된 내용을 상세하게 제시하고 있습니다.

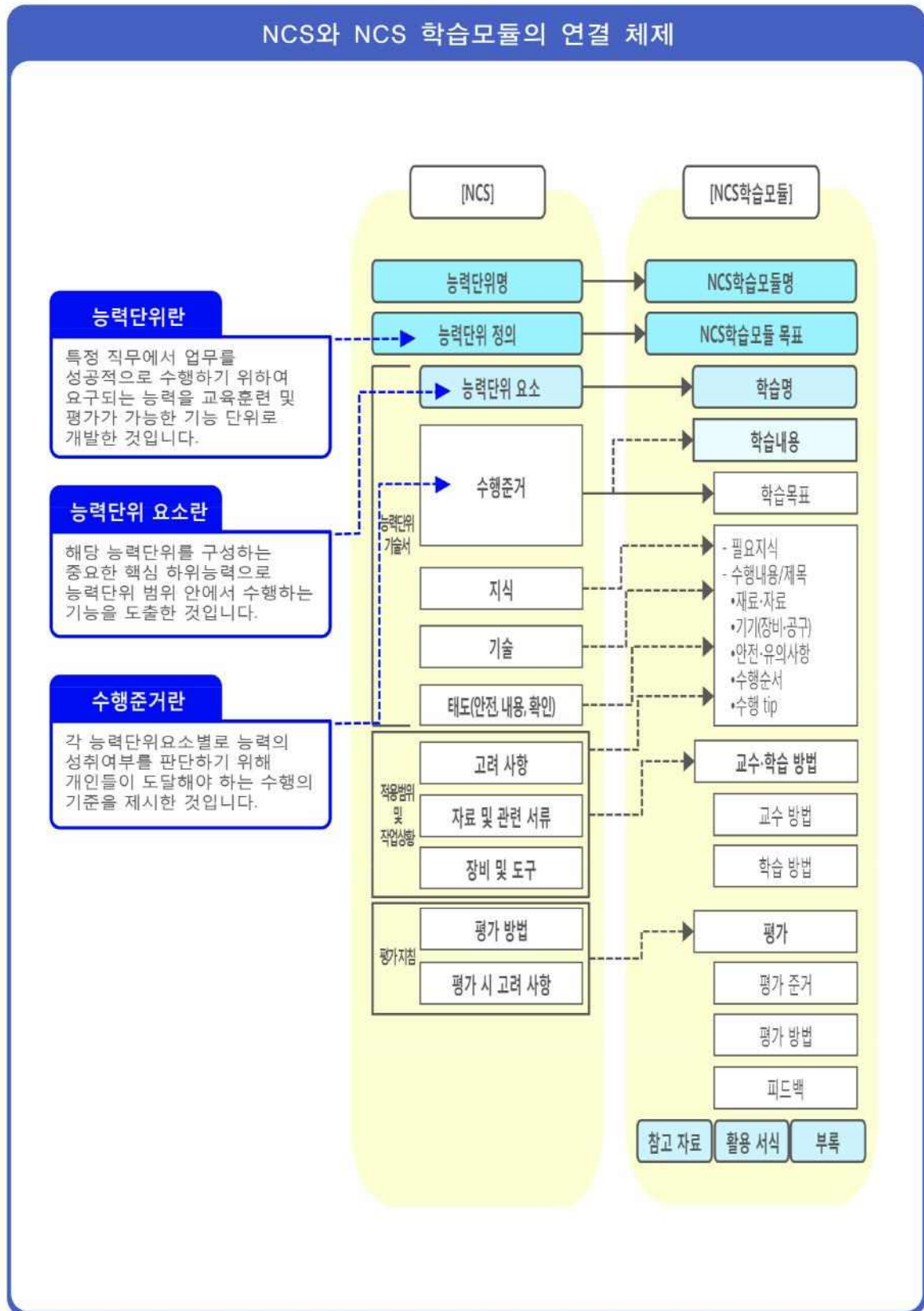


- NCS 학습모듈은 다음과 같은 특징을 가지고 있습니다.

첫째, NCS 학습모듈은 산업계에서 요구하는 직무능력을 교육훈련 현장에 활용할 수 있도록 성취목표와 학습의 방향을 명확히 제시하는 가이드라인의 역할을 합니다.

둘째, NCS 학습모듈은 특성화고, 마이스터고, 전문대학, 4년제 대학교의 교육기관 및 훈련기관, 직장교육기관 등에서 표준교재로 활용할 수 있으며 교육과정 개편 시에도 유용하게 참고할 수 있습니다.

- NCS와 NCS 학습모듈 간의 연결 체제를 살펴보면 아래 그림과 같습니다.



(2) NCS 학습모듈의 체계

- NCS 학습모듈은 1.학습모듈의 위치, 2.학습모듈의 개요, 3.학습모듈의 내용 체계, 4.참고 자료, 5.활용 서식/부록 으로 구성되어 있습니다.

1. NCS 학습모듈의 위치

- NCS 학습모듈의 위치는 NCS 분류 체계에서 해당 학습모듈이 어디에 위치하는지를 한 눈에 볼 수 있도록 그림으로 제시한 것입니다.

예시 : 이·미용 서비스 분야 중 네일미용 세분류

NCS-학습모듈의 위치

대분류 이용·숙박·여행·오락·스포츠

중분류 이·미용

소분류 아미용 서비스

세분류

헤어미용

피부미용

메이크업

네일미용

이용

능력단위	학습모듈명
네일 샵 위생 서비스	네일샵 위생서비스
네일 화장물 제거	네일 화장물 제거
네일 기본 관리	네일 기본관리
네일 랩	네일 랩
네일 팁	네일 팁
젤 네일	젤 네일
아크릴릭 네일	아크릴 네일
평면 네일아트	평면 네일아트
융합 네일아트	융합 네일아트
네일 샵 운영관리	네일샵 운영관리

학습모듈은

NCS 능력단위 1개당 1개의 학습모듈 개발을 원칙으로 합니다. 그러나 필요에 따라 고용 단위 및 교과단위를 고려하여 능력단위 몇 개를 묶어서 1개의 학습모듈로 개발할 수 있으며, NCS 능력단위 1개를 여러 개의 학습모듈로 나누어 개발할 수도 있습니다.

2. NCS 학습모듈의 개요

구 성

- NCS 학습모듈 개요는 학습모듈이 포함하고 있는 내용을 개략적으로 설명한 것으로서 **학습모듈의 목표**, **선수 학습**, **학습모듈의 내용 체계**, **핵심 용어**로 구성되어 있습니다.

학습모듈의 목표	해당 NCS 능력단위의 정의를 토대로 학습목표를 작성한 것입니다.
선수 학습	해당 학습모듈에 대한 효과적인 교수·학습을 위하여 사전에 이수해야 하는 학습모듈, 학습 내용, 관련 교과목 등을 기술한 것입니다.
학습모듈의 내용 체계	해당 NCS 능력단위요소가 학습모듈에서 구조화된 방식을 제시한 것입니다.
핵심 용어	해당 학습모듈의 학습 내용, 수행 내용, 설비·기자재 등 가운데 핵심적인 용어를 제시한 것입니다.

활 용 안 내

예시 : 네일미용 세분류의 '네일 기본관리' 학습모듈

네일 기본관리 학습모듈의 개요

학습모듈의 목표

고객의 네일 보호와 미적 요구 충족을 위하여 효과적인 네일 관리로 프리에지 형태 만들기, 큐티를 정리하기, 컬러링하기, 보습제 도포하기, 마무리를 할 수 있다.

선수 학습

네일숍 위생서비스(LM20X010401_14v2)

학습모듈의 내용체계

학습	학습 내용	NCS 능력단위 요소	
		코드번호	요소 명칭
1. 프리에지 형태 만들기	1-1. 네일 파일에 대한 이해와 활용 1-2. 프리에지 형태 파일링	1201010403_12v2.1	프리에지 모양 만들기
2. 큐티를 정리하기	2-1. 네일 기본관리 매뉴얼 이해 2-2. 큐티를 관리	1201010403_14v2.2	큐티를 정리하기
3. 컬러링하기	3-1. 컬러링 매뉴얼 이해 3-2. 컬러링 방법 선정과 작업 3-3. 쥬 컬러링 작업	1201010403_14v2.3	컬러링
4. 보습제 도포하기	4-1. 보습제 선정과 도포 4-2. 각질제거	1201010403_14v2.4	보습제 바르기
5. 네일 기본관리 마무리하기	5-1. 유분기 제거 5-2. 네일 기본관리 마무리와 정리	1201010403_14v2.5	마무리하기

핵심 용어

프리에지, 니퍼, 퓨서, 플러시, 네일 파일, 스캐어형, 스캐어 오프형, 라운드형, 오발형, 모인트형

학습모듈의 목표는

학습자가 해당 학습모듈을 통해 성취해야 할 목표를 제시한 것으로, 교수자는 학습자가 학습모듈의 전체적인 내용흐름을 파악할 수 있도록 지도하는 것이 필요합니다.

선수 학습은

교수자나 학습자가 해당 모듈을 교수 또는 학습하기 이전에 이수해야 할 학습내용, 교과목, 핵심 단어 등을 표기한 것입니다. 따라서 교수자는 학습자가 개별 학습, 자기 주도 학습, 방과 후 활동 등 다양한 방법을 통해 이수할 수 있도록 지도하는 것이 필요합니다.

핵심 용어는

학습모듈을 통해 학습되고 평가되어야 할 주요 용어입니다. 또한 당해 모듈 또는 타 모듈에서도 핵심 용어를 사용하여 학습내용을 구성할 수 있으며, 「NCS 국가 직무능력표준」 사이트(www.ncs.go.kr)에서 색인(찾아보기) 중 하나로 이용할 수 있습니다.

3. NCS 학습모듈의 내용 체계

구 성

- NCS 학습모듈의 내용은 크게 **학습**, **학습 내용**, **교수·학습 방법**, **평가** 로 구성되어 있습니다.

학습	해당 NCS 능력단위요소 명칭을 사용하여 제시한 것입니다. 학습은 크게 학습 내용, 교수·학습 방법, 평가로 구성되며 해당 NCS 능력단위의 능력단위 요소별 지식, 기술, 태도 등을 토대로 학습 내용을 제시한 것입니다.
학습 내용	학습 내용은 학습 목표, 필요 지식, 수행 내용으로 구성하였으며, 수행 내용은 재료·자료, 기기(장비·공구), 안전·유의 사항, 수행 순서, 수행 tip으로 구성한 것입니다. 학습모듈의 학습 내용은 업무의 표준화된 프로세스에 기반을 두고 실제 산업현장에서 이루어지는 업무활동을 다양한 방식으로 반영한 것입니다.
교수·학습 방법	학습 목표를 성취하기 위한 교수자와 학습자 간, 학습자와 학습자 간의 상호 작용이 활발하게 일어날 수 있도록 교수자의 활동 및 교수 전략, 학습자의 활동을 제시한 것입니다.
평가	평가는 해당 학습모듈의 학습 정도를 확인할 수 있는 평가 준거, 평가 방법, 평가 결과의 피드백 방법을 제시한 것입니다.

활 용 안 내

예시 : 네일미용 세분류의 ‘네일 기본관리’ 학습모듈의 내용

학습 1	프리에지 형태 만들기(LM1201010403_14v2.1)
학습 2	큐티를 정리하기(LM1201010403_14v2.2)
학습 3	컬러링하기(LM1201010403_14v2.3)
학습 4	보습제 도포하기(LM1201010403_14v2.4)
학습 5	네일 기본관리 마무리하기(LM1201010403_14v2.5)

학습은

해당 NCS 능력단위요소 명칭을 사용하여 제시하였습니다. 학습은 일반교과의 대단원에 해당되며, 모듈을 구성하는 가장 큰 단위가 됩니다. 또한 완성된 직무를 수행하기 위한 가장 기본적인 단위로 사용할 수 있습니다.

학습내용은

요소 별 수행준거를 기준으로 제시하였습니다. 일반교과의 '중단원'에 해당합니다.

학습목표는

모듈 내의 학습내용을 이수했을 때 학습자가 보여줄 수 있는 행동수준을 의미합니다. 따라서 일반 수업시간의 과목목표로 활용할 수 있습니다.

3-1. 컬러링 매뉴얼 이해

학습목표

- 고객의 요구에 따라 네일 폴리시 색상과 질감을 만들기 위한 베이스코트를 아주 얇게 도포할 수 있다.
- 작업 매뉴얼에 따라 네일 폴리시를 얼룩 없이 균일하게 도포할 수 있다.
- 작업 매뉴얼에 따라 네일 폴리시 도포 후 컬러 보습과 광택 부여를 위한 톱코트를 바를 수 있다.

필요 지식 /

Ⅱ 컬러링 매뉴얼

컬러링 작업 전, 이세론 또는 네일 폴리시 리무버를 사용하여 손톱표면과 큐티를 주변, 손톱 밑 부분까지 깨끗하게 유분기를 제거해야 한다. 컬러링의 순서는 Base coating 1회 → Polishing 2회 → 컬러수정 → Top coating 1회 → 최종수정의 순서로 한다. 베이스코트는 착색을 방지하고 발림성 향상을 위해 가장 먼저 도포하며 컬러링의 마지막에 컬러의 유지와 광택을 위해 톱코트를 도포한다. 네일 보강제(Nail Strengthner)를 바를 시에는 베이스코트를 도포하기 전에 사용한다.

필요지식은

해당 NCS의 지식을 토대로 해당 학습에 대한 이해와 성과를 높이기 위해 알아야 할 주요 지식을 제시하였습니다. 필요지식은 수행에 꼭 필요한 핵심 내용을 위주로 제시하여 교수자의 역할이 매우 중요하며, 이후 수행순서 내용과 연계하여 교수·학습으로 진행할 수 있습니다.

수행 내용 / 컬러링 매뉴얼 실습하기

재료·자료

- 컬러링 관련 네일 미용 자료들
- 컬러바구니, 베이스코트, 네일 폴리시, 톱코트, 오렌지우드스틱, 단자면, 폴리시리무버, 디스펜서 등

기기(장비·공구)

- 컴퓨터, 빔 프로젝터, 스크린 등

안전·유의사항

- 컬러링 재료들의 냄새를 직접적으로 맡지 않도록 유의한다.
- 컬러링 제품들이 대부분 유리병에 들어 있기 때문에 깨지지 않도록 각별히 조심한다.
- 컬러링 제품들은 상온에 마르기 때문에 개봉 후 뚜껑을 잘 닫도록 한다.

수행 순서

1) 네일 폴리시를 바르게 짚는다.

1. 손바닥에 네일 폴리시를 놓고 약지 소지를 이용하여 네일 폴리시를 짚는다.
2. 폴리시를 왼 손의 엄지와 검지로 고객의 작업손가락을 짚는다.
3. 폴리시를 왼 손의 중지 손가락을 굳게 펴서 발집대가 되도록 한다.
4. 반대편 손으로 네일 폴리시의 뚜껑을 열고 소지 손가락을 펴서 네일 폴리시를 왼 중지 손가락 위에 받쳐놓는다.
5. 다양한 형태의 폴리시를 잡아본다.

수행 tip

- 흰색이 많이 섞인 네일 폴리시의 경우는 붓의 각도를 높이 세워 빠르게 브러시 작업을 해야 붓 자국이 나지 않는다.
- 컬러링은 기본 2회 정도이나 컬러에 따른 도포량과 컬러감에 따라 1~3회 사이로 증감할 수 있다.

수행 내용은

모듈에 제시한 것 중 기술(Skill)을 습득하기 위한 실습 과제로 활용할 수 있습니다.

재료·자료는

수행 내용을 수행하는데 필요한 재료 및 준비물로 실습 시 필요 준비물로 활용할 수 있습니다.

기기(장비·공구)는

수행 내용을 수행하는데 필요한 기본적인 장비 및 도구를 제시하였습니다. 제시된 기기 외에도 수행에 필요한 다양한 도구나 장비를 활용할 수 있습니다.

안전·유의사항은

수행 내용을 수행하는데 안전상 주의해야 할 점 및 유의 사항을 제시하였습니다. 수행 시 유념해야 하며, NCS의 고려사항도 추가적으로 활용할 수 있습니다.

수행 순서는

실습과제의 진행 순서로 활용할 수 있습니다.

수행 tip은

수행 내용에서 수행의 수월성을 높일 수 있는 아이디어를 제시하였습니다. 따라서 수행tip은 지도상의 안전 및 유의 사항 외에 전반적으로 적용되는 주의점 및 수행과제 목적에 대한 보충설명, 추가사항 등으로 활용할 수 있습니다.

학습3 교수·학습 방법

교수 방법

- 컬러링 제품의 성분과 컬러별 정도의 차이, 베이스코트와 톱코트의 역할, 폴리시 짚는 방법, 큐어링 시간 등의 내용을 회선 자료와 함께 설명한다.
- 서식지를 활용하여 네일 컬러링 방법을 그림으로 그려 보게 한 뒤, 다양한 컬러링의 매뉴얼을 그려서 숙지하도록 한다.
- 셀 컬러링 시 주의사항을 계속 숙지시키도록 하며, 큐어링 시간에 대해 작성하도록 한다.

학습 방법

- 컬러링을 위한 재료의 필요성과 사용방법을 숙지하고 컬러링 매뉴얼 과정에 맞추어 작업 내용을 이해한다.
- 컬러링의 다양성에 대한 용어를 숙지하고 진행과정에 맞추어 내용을 직접한다.
- 셀 컬러링 시 적합한 큐어링 시간을 선택해서 큐어링 해본다.

교수·학습 방법은

학습목표를 성취하는데 필요한 교수 방법과 학습 방법을 제시하였습니다.

교수 방법은

해당 학습활동에 필요한 학습내용, 학습내용과 관련된 학습 자료명, 자료 형태, 수행내용의 진행 방식 등에 대하여 제시 하였습니다. 또한 학습자의 수업참여도를 제고하기 위한 방법 및 수업진행상 유의사항 등도 제시하였습니다. 선수학습 이 필요한 학습을 학습자가 숙지하였는지 교수자가 확인하 는 과정으로 활용할 수도 있습니다.

학습 방법은

해당 학습활동에 필요한 학습자의 자기주도적 학습 방법 을 제시하였습니다. 또한 학습자가 숙달해야 할 실기능력과 학습과정에서 주의해야 할 사항 등으로 제시하였습니 다. 학습자가 학습을 이수하기 전에 반드시 숙지해야 할 기본 지식을 학습하였는지 스스로 확인하는 과정으로 활 용할 수 있습니다.

학습3 평가

평가 준거

- 평가자는 학습자가 학습 목표 및 평가 항목에 제시되어 있는 내용을 성공적으로 수행하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습내용	평가항목	성취수준		
		상	중	하
킬러링 매뉴얼 이해	- 고객의 요구에 따라 네일 폴리시 색상의 질착을 막기 위한 베이스코트를 아주 얇게 도포할 수 있다.			
	- 작업 매뉴얼에 따라 네일 폴리시를 일찍 얹어 균일하게 도포할 수 있다.			
	- 작업 매뉴얼에 따라 네일 폴리시 도포 후 컬러 보호와 광택 부여를 위한 톱코트를 바를 수 있다.			

평가 방법

- 작업장 평가

학습내용	평가항목	성취수준		
		상	중	하
킬러링 매뉴얼 이해	- 고객의 요구에 따라 네일 폴리시 색상의 질착을 막기 위한 베이스코트를 아주 얇게 도포할 수 있다.			
	- 작업 매뉴얼에 따라 네일 폴리시를 일찍 얹어 균일하게 도포할 수 있다.			
	- 작업 매뉴얼에 따라 네일 폴리시 도포 후 컬러 보호와 광택 부여를 위한 톱코트를 바를 수 있다.			

피드백

- 작업장 평가
 - 작업 결과물을 확인하여 수정사항을 제시하고 수정 부분을 인지하도록 한다.

평가는

해당 NCS 능력단위 평가방법과 평가 시 고려 사항을 준용하여 작성하였습니다. 교수자 및 학습자가 평가항목 별 성취수준을 확인하는데 활용할 수 있습니다.

평가 준거는

학습자가 해당 학습을 어느 정도 성취하였는지를 평가하기 위한 기준을 제시하고 있습니다. 학습목표와 연계하여 단위수업 시간에 평가항목 별 성취수준을 평가하는데 활용할 수 있습니다.

평가 방법은

NCS 능력단위의 평가방법을 준용하였으며, 평가 준거에 따른 평가방법을 2개 이상 제시하였습니다. 평가방법으로는 포트폴리오, 문제해결 시나리오, 서술형 시험, 논술형 시험, 사례연구, 평가자 체크리스트, 작업장 평가 등이 있으며, NCS의 능력단위 요소 별 수행 수준을 평가하는데 가장 적절한 방법을 선정하여 활용할 수 있습니다.

피드백은

평가 후에 학습자들에게 평가 결과를 피드백하여 부족한 부분을 알려주고, 학습 결과가 미진한 경우, 해당 부분을 다시 학습하여 학습목표를 달성하는 데 활용할 수 있습니다.

4. 참고 자료

참고자료

- 김미원(2011). 『Nail Study』. 서울: 사)한국네일저서서비스협회.
- 민동경(2015). 『미용·네일·관리』. 서울: 예문사.
- 박은주(2014). 『네일미용』. 서울: 정담미디어.

참고자료는

해당 학습모듈의 필요지식에 대한 출처와 인용한 참고 자료 및 사이트를 제시하였습니다.

5. 활용 서식/부록

활용서식

프리페이지 형태 실습지

1. 프리페이지 형태의 이해

모양	이름	특징
	이름: Square nail	- 강한 느낌의 사각형태 - 네일의 양끝 모서리 부분이 90° 사각의 형태이다. - 발톱의 형태 활용 - 내인성 발톱의 보정시에 적용

활용서식은

평가 서식, 실습시트 등 교수학습 시 활용 가능한 다양한 서식들로 구성하였습니다. 과제 진행에서 평가에 이르기까지 필요한 서식을 해당 학습모듈의 특성에 맞춰 개발하거나 기존의 양식을 활용하여 제시하였습니다.

부록

네일 기본관리 도구와 재료 목록

목록	비고	준비
위생가운	환식	작업자 착용
위생 마스크	환식	작업자 착용
보호안경	두꺼운 렌즈 (안경으로 대체 가능)	작업자 착용
재활정리갑	세척, 색상 무관	작업대

부록은

활용서식 이외에 교수학습과정에서 참고할 수 있는 자료가 있는 경우 제시하였습니다.

[NCS-학습מוד의 위치]

대분류	전기·전자	
중분류	전자 기기 개발	
소분류	로봇 개발	

세분류	능력단위	학습מוד명
로봇 하드웨어 설계		
로봇 기구 개발	로봇 작업 요구사항 분석	로봇 작업 요구 사항 분석 및 소프트웨어 아키텍처 설계
로봇 소프트웨어 개발	로봇 소프트웨어 아키텍처 설계	
로봇 지능 개발	로봇 액추에이터 제어 소프트웨어 개발	로봇 액추에이터 제어 소프트웨어 개발
로봇 유지보수	로봇 센서 인터페이스 개발	로봇 센서 인터페이스 개발
	로봇미들웨어 개발	로봇미들웨어 설계
	로봇용 UX UI 개발	로봇용 UX UI 개발
	로봇 콘텐츠 소프트웨어 개발	로봇 콘텐츠 소프트웨어 개발
	로봇 소프트웨어 시험평가	로봇 소프트웨어 시험평가
	로봇 운영 소프트웨어 개발	로봇 운영 및 통합 소프트웨어 개발
	로봇 시스템 통합 소프트웨어 개발	
	로봇 시뮬레이터 개발	로봇 시뮬레이터 개발
	로봇 소프트웨어 유지보수	로봇 소프트웨어 유지보수
	경로 계획 소프트웨어 개발	경로 계획 소프트웨어 개발

세분류

로봇 소프트웨어 개발	능력단위	학습모듈명
	역기구학 소프트웨어 개발	역기구학 및 동역학 소프트웨어 개발
	동역학 소프트웨어 개발	
	작업지능 소프트웨어 개발	작업지능 소프트웨어 개발
	이동지능 소프트웨어 개발	이동지능 소프트웨어 개발
	인지지능 소프트웨어 개발	로봇 모션 제어 및 지능소프트웨어 개발

차 례

학습모듈의 개요	1
학습 1. 단위 기능 시험평가하기	
1-1. 단위 기능 시험평가	3
• 교수·학습 방법	28
• 평가	30
학습 2. 통합 기능 시험평가하기	
2-1. 통합 기능 시험평가	33
• 교수·학습 방법	53
• 평가	54
학습 3. 신뢰성 시험평가하기	
3-1. 신뢰성 시험평가	57
• 교수·학습 방법	83
• 평가	84
참고 자료	87
활용 서식	88

로봇 소프트웨어 시험평가 학습모듈의 개요

학습모듈의 목표

로봇 소프트웨어를 다양한 경우의 수에 대한 확인을 하여 잠재적으로 가지고 있을 수 있는 결함을 식별할 수 있다.

선수학습

프로그래밍 언어, 로봇공학 개론, 전기전자공학 개론, 메카트로닉스

학습모듈의 내용 체계

학습	학습 내용	NCS 능력단위 요소	
		코드번호	요소 명칭
1. 단위기능 시험평가하기	1-1. 단위기능 시험평가	1903080310_14v1.1	단위기능 시험평가하기
2. 통합기능 시험평가하기	2-1. 통합기능 시험평가	1903080310_14v1.2	통합기능 시험평가하기
3. 신뢰성 시험평가하기	3-1. 신뢰성 시험평가	1903080310_14v1.3	신뢰성 시험평가하기

핵심 용어

소프트웨어 테스트, 단위 시험, 통합 시험, 신뢰성 시험, 정적 분석, 동적 분석, 테스트 케이스, 자동화 도구

학습 1 단위기능 시험평가하기

학습 2 통합기능 시험평가하기

학습 3 신뢰성기능 시험평가하기

1-1. 단위기능 시험평가

학습 목표

- 단위기능의 정상적 수행 여부를 판단할 수 있는 시험 절차를 작성할 수 있다.
- 시험 절차서에 따라 단위기능을 구현한 함수의 동작이 적절한지 판단할 수 있다.
- 단위기능을 시험한 결과를 보고서로 작성할 수 있어야 한다.

필요 지식 /

① 프로그램 언어

프로그램 언어는 컴퓨터 시스템을 동작시키는 프로그램을 작성하기 위한 언어이며, 사람이 사용하기 편리한 정도에 따라 저급언어와 고급언어로 분류된다.

<표 2-1> 프로그램 언어의 종류

구분	프로그램 언어 예	특징
저급언어	기계어, 어셈블리어	기계 중심적인 언어 컴퓨터 시스템마다 달라 호환이 어려움.
고급언어	C, C++, ADA, JAVA 등	인간이 사용하는 자연어와 닮음. 대부분의 언어가 고급언어임.

이 중 로봇 개발 시 주로 많이 사용되는 프로그램 언어는 C, C#, C++, JAVA, Python 등이다.

1. C 언어

C는 1972년 켄 톰슨과 데니스 리치가 벨 연구소에서 일할 당시 새로 개발된 유닉스 운영 체제에서 사용하기 위해 개발한 프로그래밍 언어이다. 오늘 날 많은 운영체제의 커널이 C 언어로 작성되었고, C 표준 라이브러리가 국제표준화기구에 의해 채택되었으며, C++, C# 등의 확장형 언어의 뿌리가 되었다.

2. JAVA

자바는 썬 마이크로시스템즈의 제임스 고슬링(JAMES GOSLING)과 다른 연구원들이 개발한 객체 지향적 프로그래밍 언어이며, 무료로 제공되고 있다. 처음에는 가전제품 내에 탑재해 동작하는 프로그램을 위해 개발했지만 현재 웹 애플리케이션 개발에 가장 많이 사용하는 언어 가운데 하나이고, 모바일 기기용 소프트웨어 개발에도 널리 사용하고 있다.

자바의 개발자들은 유닉스 기반의 배경을 가지고 있었기 때문에 문법적인 특성은 파스칼이 아닌 C 언어와 비슷하다. 자바를 다른 컴파일 언어와 구분 짓는 가장 큰 특징은 컴파일된 코드가 플랫폼 독립적이라는 점이다. 자바 컴파일러는 자바 언어로 작성된 프로그램을 바이트코드라는 특수한 바이너리 형태로 변환한다. 바이트코드를 실행하기 위해서는 JVM(자바 가상 머신, java virtual machine)이라는 특수한 가상 머신이 필요한데, 이 가상 머신은 자바 바이트코드를 어느 플랫폼에서나 동일한 형태로 실행시킨다. 때문에 자바로 개발된 프로그램은 CPU나 운영체제의 종류에 관계없이 JVM을 설치할 수 있는 시스템에서는 어디서나 실행할 수 있으며, 이 점이 웹 애플리케이션의 특성과 맞아 떨어져 폭발적인 인기를 끌게 되었다.

3. PYTHON

파이썬(PYTHON)은 1991년 프로그래머인 귀도 반 로섬(GUIDO VAN ROSSUM)이 발표한 고급 프로그래밍 언어로, 플랫폼 독립적이며 인터프리터식, 객체 지향적, 동적 타이핑(dynamically typed) 대화형 언어이다.

파이썬은 비영리의 파이썬 소프트웨어 재단이 관리하는 개방형, 공동체 기반 개발모델을 가지고 있다. C언어로 구현된 C파이썬 구현이 사실상의 표준이다.

② 소프트웨어 시험 관련 용어

1. 소프트웨어 시험

IEEE STD 829 에서는 소프트웨어 시험의 정의를 아래와 같이 정의하고 있다.

“요구된 상태와 현재 개발된 상태 사이의 차이점(즉, 버그)을 발견하기 위해 소프트웨어를 분석하고 평가하는 프로세스(The process of analyzing a software item to detect the differences between existing and required conditions(that is, bugs) and to evaluate the features of the software item)”

2. 확인(verification)과 검증(validation)

(1) 확인(verification)

확인(verification)은 한 마디로 하면 제품을 올바르게 만드는지?(Are we building the product right?)를 확인하는 과정이다. 제품이 설계 사양에 부합되게 제작되고 있는지를 확인한다.

(2) 검증(Validation)

검증은 올바른 제품을 만드는가?(Are we building the right product?)라는 관점에서 바라보며 설계 사양이 사용자의 요구 사항에 부합되는지를 검증한다.

3. 결함 관련

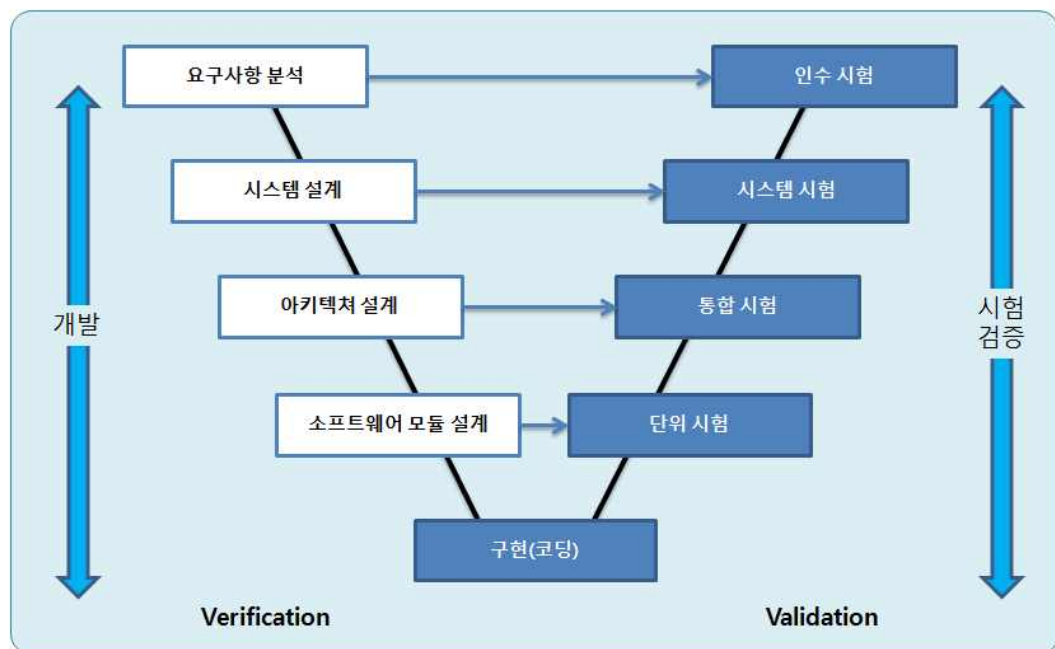
- 에러, 오류(error): 요구 사항을 잘못 이해하여 발생하는 실수
- 결함(fault): 에러가 원인이 되어 프로그램에 반영된 결과
- 실패(failure): 결함이 있는 부분이 실행될 때 발생하는 현상

③ 소프트웨어 시험 종류

소프트웨어 시험에는 시험의 단계별 혹은 시험 목적이나 시험 방법 등에 따라 매우 다양하게 구분할 수 있다.

1. 시험 단계(test level)

소프트웨어 개발 수명 주기 V-모델의 단계에 따라 단위 시험, 통합 시험, 시스템 시험, 인수 시험 등으로 구분한다. 시험 검증은 개발의 역순으로 진행한다.



[그림 1-1] V-모델

<표 1-2> 단계별 시험의 종류

순번	구분	내용	검증
1	단위 시험	각 컴포넌트에 대한 시험으로 세부 구현 클래스와 함수에 대한 검증	알고리즘, 로직, 정확도 및 정밀도, 예외 처리 등
2	통합 시험	컴포넌트 간 기능 연동을 위한 검증	매개변수 전달, 전역데이터 손상, 동작 순서 오류 등
3	시스템 시험	기능 시험 및 비기능 시험을 포함한 배포 환경에 대한 검증	인터럽트 핸들링, 데이터베이스/자원 연결 문제, 자체 고장 탐지 등
4	인수 시험	요구 사항 충족 확인	기능/성능/운용 요구 조건 등

2. 시험 방법 - 코드 수행 여부에 따른 구분

실제로 작동하는 런타임 소프트웨어로 시험을 하느냐 아니면 문서나 코드와 같이 정적인 자료를 가지고 검증을 하느냐에 따라 정적 시험과 동적 시험으로 분류할 수 있다.

(1) 정적 시험

프로그램 코드에 대한 정적 시험은 사람 중심의 코드리뷰 방법과 자동화 시스템에 의한 정적 분석 방법으로 크게 나뉜다.

<표 1-3> 정적 시험 방법

방법	특징
코드리뷰	개발 과정에서 가장 쉽게 적용할 수 있는 방법 리뷰어(참여하는 사람)의 능력이나 참여 정도에 따라 결과 편차 큼. 피어 리뷰, 패스 어라운드, 팀 리뷰, 워크스루, 코드 인스펙션 등
정적 분석	자동화된 도구에 의해 수행됨. 코드 복잡도 계산, 코딩 규칙 위배 및 사용하지 않는 데드코드 검출 결과의 편차가 거의 없음. 오픈소스 도구 및 상용화 버전 있음.

(2) 동적 시험

동적 시험은 코드를 실행하면서 수행하는 시험으로 코드 실행률(code coverage)이 주요 시험의 지표가 된다.

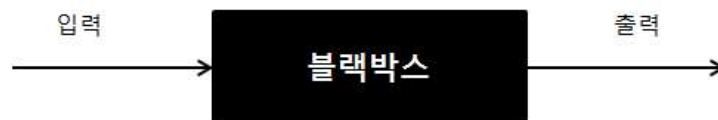
3. 시험 방법 - 박스 구분

(1) 블랙박스 시험

블랙박스 시험은 사용자의 시각으로 평가하는 방법으로 테스트팀 위주로 이루어지는 시험이며 모든 단계의 시험이 다 가능하나 일반적으로 시스템 시험과 인수 시험이 대상이 된다.

이 테스트는 내부 구조나 작동 원리를 모르는 상태에서 입력과 출력값만으로 확인하는 시험이다. 다만 해당 프로그램이 어떠한 기능을 가지고 있으며 어떻게 동작하는가를 정의한 요구 사항 명세서나 설계 명세서를 참조하여 테스트한다.

이 시험을 통하면 기능상의 문제나 인터페이스 문제 혹은 database와 관련된 에러를 발견할 수 있다. 그러나 프로그램의 내부 로직을 모르는 상태로 시험을 하기 때문에 결함의 요인을 발견하기는 화이트박스 시험보다 상대적으로 어렵다.

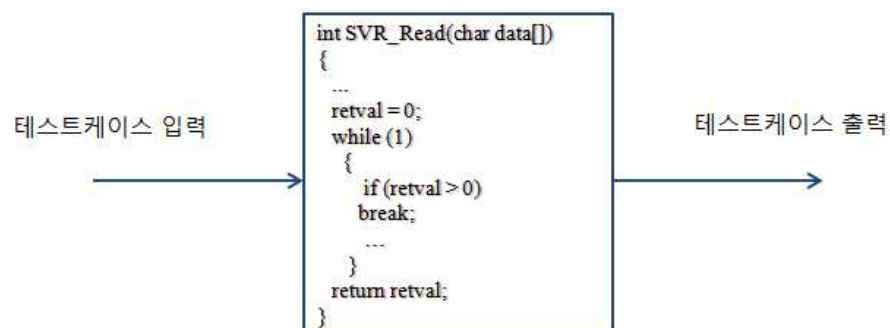


[그림 1-2] 블랙박스 시험 개념도

(2) 화이트박스 시험

화이트박스 시험은 시스템의 내부 구조를 이해하고, 내부 로직 구조를 확인하는 시험으로 프로그램 코드들의 논리적인 모든 경로를 검사하는 방법이다. 개발팀 주도로 이루어지는 시험이며 보통 단위 시험과 통합 시험이 대상이 된다.

화이트박스 시험은 프로그램을 실행시키지 않고도 프로그램 구현 논리나 조건에 관한 사항, 내부 자료 구조에 대한 것들을 시험할 수 있다.



[그림 1-3] 화이트박스 시험 개념도

4. 시험 유형

(1) 기능 시험(functional test)

소프트웨어가 가지고 있는 본연의 목적에 해당되는 기능에 대한 시험을 의미한다. 해당 소프트웨어의 기능은 요구 사항 명세서나 설계 명세서 등에 정의되어 있으며 모든 시험 단계에서 수행될 수 있다.

(2) 비기능 시험(non-functional test)

비기능 시험은 성능 시험, 장애 시험, 안정성 시험, 확장성 시험, 신뢰성 시험 등 소프트웨어가 가지고 있어야 하는 기능 외에 소프트웨어 제품의 특성에 대한 시험이라고 생각할 수 있다.

<표 1-4> 비기능 시험의 종류

시험 구분	정의
성능 시험	시스템에 부하를 주면서 요구하는 성능 지표를 확인하는 시험
장애 시험	시스템에 장애가 생겼을 때 이의 감지와 장애 복구 능력을 검증하는 시험
안정성 시험	시스템이 오랫동안 운영이 되더라도 문제가 없는지 검증하는 시험
확장성 시험	시스템을 증설함에 따라 용량이 선형적으로 증가하는지 확인하는 시험

(3) 확인 시험(confirmation test)

확인 시험은 다양한 시험 과정에서 발견된 결함을 해결한 후에 결함이 제대로 해결되었는지 검증하는 과정이다.

(4) 회귀 시험(regression test)

회귀 시험은 앞 단계에서 정상적으로 수행된 시험을 반복하는 과정이다. 이 시험은 이미 시험이 수행된 코드가 아닌 다른 부분의 결함 해소나 소프트웨어의 변경 혹은 추가 진행 등으로 기존에 잘 작동하던 기능이 영향을 받지 않는지 검증하는 과정이다.

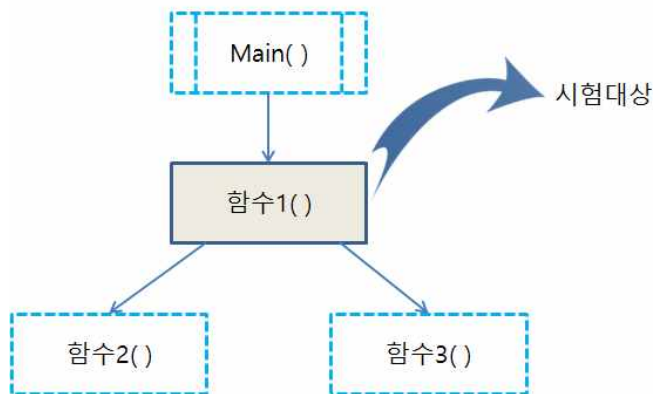
소프트웨어 시험에는 이 외에도 매우 다양한 종류의 시험들이 있다.

요즘과 같이 기능이 많고 복잡한 소프트웨어는 버그나 결함이 전혀 없는 상태로 만드는 것이 거의 불가능하다. 소프트웨어 시험은 작은 비용과 시간을 투입하여 최대한 효율적으로 최대한 많은 결함을 찾아내는 것을 목표로 해야 한다. 따라서 전체 시험을 계획할 때 무슨 시험을 언제 어떠한 방법으로 수행할지를 정하는 것이 매우 중요하다.

④ 단위기능 시험

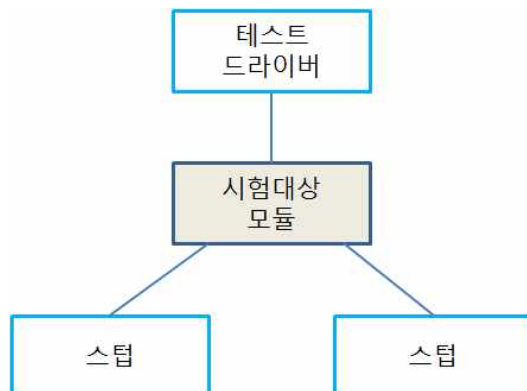
1. 단위 시험 정의

단위 시험은 프로그램을 구성하는 가장 작은 구성 요소(클래스나 함수 단위)를 대상으로 하며, 소스 코드의 각 함수가 프로그래머의 의도대로 정확히 작동하는지를 확인 검증하는 시험이다. 시험의 방법은 아래와 같이 시험 대상 함수를 다른 함수들로부터 분리시켜 시험을 한다.



[그림 1-4] 시험 대상 함수 분리

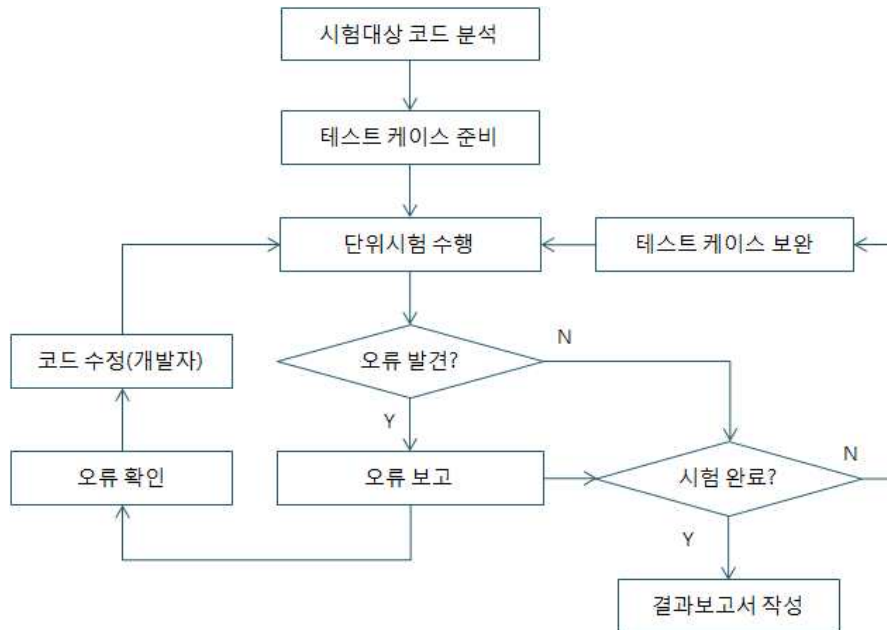
그러나 시험을 실시할 대상 함수가 바로 실행되지 않는 경우가 있기 때문에 테스트 드라이버를 만들어 시험 대상 모듈을 호출해야 할 경우도 있고, 하나의 모듈을 시험한다고 해도 하나의 모듈이 다른 모듈을 호출할 경우 시험 대상이 되는 모듈은 완전하게 실행되지 않을 수 있어 시험 대상 모듈이 호출하는 모듈인 스텝을 가상으로 만들어야 할 경우도 있다. 따라서 단위 시험을 수행하기 위해서는 아래와 같이 필요에 따라 테스트 드라이버와 스텝을 만들어 시험해야 한다. 테스트 케이스를 작성할 때 이에 대한 계획도 수립하여야 한다.



[그림 1-5] 스텝과 테스트 드라이버

2. 단위 시험 수행 절차

단위 시험의 수행 절차는 아래와 같다.



출처 : <http://blog.naver.com/suresofttech/220694868279> 2016. 07. 03. 검색

[그림 1-6] 단위 시험 절차

(1) 시험 환경 구축

소프트웨어 시험을 하기 위해서는 시험 환경을 구축하여야 하며 이를 위해서는 개발 환경을 알아야 한다. 하드웨어 아키텍처, 운영체제, 개발 프로그래밍 언어 및 통합개발 환경과 사용 컴파일러 등에 대해 조사하여 이에 맞게 시험 환경을 구축하여야 한다.

(2) 기능 및 코드 분석

해당 소프트웨어가 가지고 있는 기능과 코드가 어떤 방식으로 구현되어 있는지를 분석하여 테스트 케이스 작성 시 참고하여야 한다. 이를 위해 요구 사항 분석 자료와 설계 자료를 참조한다.

(3) 테스트 케이스 작성

실제 테스트를 수행할 시 시험 대상 함수에 입력값을 주고, 그 실제 출력값이 예상 출력값과 같은지를 비교하는 방식으로 시험을 진행한다. 그러기 위해서는 입력값과 예상되는 출력값을 미리 정하게 되는데 이것을 테스트 케이스라고 한다. 또 테스트 드라이버와 스텝에 대한 정의도 수행하여야 한다.

(4) 결과보고서 작성

모든 시험이 완료된 후에는 이를 종합 정리하여 결과보고서를 작성하여야 한다. 전체 시험 계획에 식별되어 있는 많은 시험 중 이 보고서를 근거로 단위 시험 완료를 확인한다.

⑤ 단위 시험 절차서

1. 시험 계획서

소프트웨어 개발이 완료되고 시험 단계에 들어가기 전에 소프트웨어 시험에 대한 계획을 수립한다. IEEE STD 829에서는 마스트 테스트 플랜(master test plan)과 레벨 테스트 플랜(level test plan)을 규정하고 있다. 이 계획에는 시험 대상 소프트웨어 품목, 시험 환경, 수행될 시험 내용, 시험 방법과 범위가 정의되어 있다. 규모가 큰 개발 프로젝트의 경우는 시험 계획서가 먼저 나오고 이 계획을 바탕으로 시험 절차서를 작성한다.

<표 1-5> 시험 계획서 (사례)

ooo 장비	
소프트웨어 시험 계획서	
(Software Test Plan)	
1. 개요	- 장비 설명, 개발 일정 등
2. 시험 환경	- 시험 대상 품목, 수행될 시험, 시험 조직 등
3. 시험 식별	- 시험 수준, 시험 분류, 시험 조건, 시험 계획 등
4. 일정	- 시험 일정 계획 및 장소
5. 요구 사항 확인 방법	- 요구 사항 추적성, 식별자 정의
	:

2. 시험 대상 프로그램 파악

시험을 수행하기 전에 처음으로 해야 하는 것은 시험 대상 프로그램을 파악하는 것이다. 만일, 개발자가 단위 시험을 수행한다면 이 과정이 필요 없겠지만 개발자가 아닌 제 3자가 시험을 수행한다면 반드시 필요한 과정이다. 대상 프로그램의 목적과 기능은 무엇인지? 어떤 하드웨어 플랫폼에서 동작하는지? 개발환경은 어떠했는지? 등에 대해 구체적으로 알아야 시험을 정확히 수행할 수 있다.

시험 대상 프로그램을 파악하는 가장 좋은 방법은 아래와 같은 개발과 설계 단계에서의 산출물을 확인하는 것이다.

- 운용 개념 기술서
- 개발 계획서
- 요구 사항 명세서
- 설계 기술서
 - 사용 사례 다이어그램(usecase diagram)
 - 클래스 다이어그램(class diagram)
 - 순차 다이어그램(sequence diagram) 등

이를 통해서, 아래와 같은 내용을 파악한다.

(1) 프로그램 사용 환경

<표 1-6> 프로그램 사용 환경 (사례)

구분	내용 (예)
프로그램 명	AAA ver 0.1
사용 체계	BBB 제어 컨트롤러 시리즈
적용 프로세서	DSP TMSXXXXXX
적용 운영체제/BIOS	DSP/BIOS
:	:

(2) 개발환경 및 코드 정보

<표 1-7> 프로그램 개발환경 및 코드 정보 (사례)

구분	내용 (예)
칩(CPU/MCU)	TMS320C6713
개발언어	C
개발환경	Code Composer Studio
컴파일러	c6x.exe
라인 수	00,000 line
파일 수	00개
함수 수	00개
:	:

(3) 시험 대상 항목 추출

시험 대상 프로그램의 함수 목록을 추출한다.

<표 1-8> 함수 목록 (사례)

모듈 명	번호	함수 명 (예)	설명(예)
RunMain	1	GetCmd	명령값을 가지고 온다.
	2	SaveData	입력된 데이터를 저장한다.
	3	doCommand	명령을 실행한다.
	4	SetSpeed	속도값을 설정한다.
	5	GetSpeed	속도값을 읽어 들인다.
	6	Emergency_Stop	긴급 중지한다.
	:	:	:
FlashCmd	21	InitFlash	Flash ROM을 초기화한다.
	22	BurnFlash	Flash ROM에 데이터를 굽는다.
	23	EraseFlash	Flash ROM을 지운다.
	:	:	:

3. 단위 시험 절차서 작성

시험 절차서에는 아래와 같은 사항이 포함되어야 하며, 적절히 테일러링해서 사용할 수 있다.

- 시험 개요
 - 시험 목적과 시험 항목
 - 시험 환경 및 필요 장비
- 개발환경
- 테스트 케이스
 - 시험 입력 자료
 - 예상 결과
 - 평가 기준
- 시험의 순서
- 요구 사항 추적성 매트릭스

(1) 요구 사항 추적성 매트릭스

설계 단계에서 최초 시스템의 요구 사항을 하위 단계에 맞는 요구 사항으로 도출해서 구현해야 하며 각 단계에서 시험 시에는 그 단계에 맞는 요구 사항을 확인하여 시험 하여야 한다.

각 단계에서 필요한 요구 사항과 시험 항목을 매트릭스로 표현해 그 추적성을 확인하는 표를 요구 사항 추적성 매트릭스(requirements traceability matrix)라고 한다. 하위 단계의 요구 사항 추적성 매트릭스는 상위의 요구 사항 추적성 매트릭스의 일부가 될 수 있다.

요구 사항 추적성 매트릭스는 다양한 형태로 작성될 수 있으나 요구 사항과 테스트 케이스는 반드시 들어가야 한다.

Requirements Traceability Matrix		
Show how each requirement is documented in the Design Specification and what Test Cases have been developed to test each requirement.		
Requirement	Design Specification	Test Case Step
3.1.1. A 300 megahertz (MHz) processor (or faster).	3.1.1. A 300 megahertz (MHz) processor (or faster).	6.1. (TC #1), Step # 1 The operating computer has a 300 megahertz (MHz) processor (or faster).
3.1.2. At least 128 MB of RAM.	3.1.2. At least 128 MB of RAM.	6.1. (TC #1), Step # 3 The operating computer has at least 128 MB of RAM.
3.1.3. At least 260 megabytes of free hard disc space.	3.1.3. At least 260 megabytes of free hard disc space.	6.1. (TC #1), Step # 4 The operating computer has at least 260 megabytes of free hard disc space.
3.1.4. KFCComm can use local or networked printers.	3.1.4. The KFCComm spreadsheet can use local or networked printers.	6.1. (TC #1), Step # 5 KFCComm can use local or networked printers.
3.2.1. The operating system is MS Windows 2000, XP, Vista or Windows 7.	3.2.1. The operating system required for use is MS Windows 2000, MS Windows XP or MS Vista.	6.1. (TC #1), Step # 6 The operating system installed for use is either MS Windows 2000 or MS Windows XP, MS Vista or Windows 7.
3.2.2. The appropriate spreadsheet template file is present.	3.2.2. The appropriate spreadsheet template file is present.	6.1. (TC #1), Step # 7 The appropriate spreadsheet template file is present.
3.2.3. The appropriate ActiveX controls are properly installed and registered.	3.2.3. KFCComm.ocx and KFScale1b.ocx are properly installed and registered.	6.1. (TC #1), Step # 9 The appropriate ActiveX controls are properly installed and registered.
3.2.4. Microsoft Excel, Version 2000 (32bit) or higher installed.	3.2.4. Microsoft Excel, Version 2000 or higher is installed.	6.1. (TC #1), Step # 14 Microsoft Excel, Version 2000 (32bit) or higher is installed.
3.2.5. All documentation required to operate and maintain the KFCComm spreadsheet template is present.	3.2.5. All documentation required to operate and maintain the KFCComm spreadsheet template is present.	6.1. (TC #1), Step # 15 All documentation required to operate and maintain the KFCComm spreadsheet template is present.
3.3.1. All required ExcelSafe files are loaded.	3.3.1. All required ExcelSafe files are loaded.	6.2. (TC #2), Step # 1 All required ExcelSafe files are loaded.
3.3.2. ExcelSafe is Version 2.1 or higher.	3.3.2. ExcelSafe is Version 2.1 or higher.	6.2. (TC #2), Step # 12 ExcelSafe is Version 2.1 or higher.
3.3.3. ExcelSafe has the expected security configuration.	3.3.3. ExcelSafe has the expected security configuration.	6.2. (TC #2), Step # 13 ExcelSafe has the expected security configuration.
3.3.4. KFCComm is properly installed in ExcelSafe.	3.3.4. KFCComm is properly installed in ExcelSafe.	6.3. (TC #3), Step # 2 KFCComm is properly installed in ExcelSafe.
October 1, 2013 There is a red indicates gaps in the traceability matrix Page 1 of 11		

출처 : <http://www.ofnissystems.com/products/fastval/details/traceability-matrix/> 2016. 08. 03. 스크린샷.

[그림 1-7] 요구 사항 추적성 매트릭스 (예시)

(2) 테스트 케이스

단위 시험을 준비하는 절차 중 가장 많은 시간이 소요되고 시험 결과 도출에 가장 중요한 절차가 테스트 케이스 작성이다. 테스트 케이스를 작성하기 위해서는 코딩을 이해할 수 있는 지식과 대상 프로그램의 원리와 구조에 대해서도 어느 정도 이해가 있어야 한다.

(가) 테스트 케이스 작성 항목

테스트 케이스를 작성할 경우 보통 아래와 같은 항목들로 구성되나 필요 없는 항목은 삭제하거나 더 필요한 항목을 추가하여도 된다.

- Test Case ID
- Test Case 에 대한 상세 설명
- 테스트 절차 / 실행 순서
- 관련 요구 사항
- 작성자
- Pass / Fail
- 기타 특이 사항

(나) 좋은 테스트 케이스 작성법

좋은 테스트 케이스 작성법은 아래와 같다.

- 가능한 한 한 번에 1개의 기능을 시험하게 작성할 것
- 포지티브 시나리오와 네거티브 시나리오를 전부 커버할 수 있도록 작성할 것
- 추상적이지 않고 명확하고 쉽게 이해할 수 있도록 작성할 것
- 목적을 명확히 할 것
- 필요 없는 절차 작성하지 말도록 할 것
- 요구 사항 추적성을 유지하도록 작성할 것
- 반복 시험할 수 있도록 필요하다면 다시 사용 가능하게 작성할 것

⑥ 자동화 도구를 이용한 단위 시험

요즘처럼 소프트웨어가 기능이 많아지고 복잡해지면 사람이 모든 시험을 수행하고 관리하기가 쉽지 않아 자동화 도구를 사용하는 게 일반적이다. 자동화 도구에 대해서는 ‘학습 내용 3-2’ 를 참조한다.

자동화 도구는 시험 수행과 관리를 편리하게 해주는 도구이지 시험에 대한 판단까지 해주는 만능 도구가 아니다. 자동화 도구는 프로그램 코딩 규칙이나 메모리 누수 같은 오류는 식별해 줄 수 있지만, 프로그램상의 로직까지는 판단해 주지 못한다. 이는 시험자가 확인해서 찾아내야 할 몫이다.

수행 내용 / 단위 기능 시험 수행하기

재료 · 자료

- 노트 및 필기도구
- 시험 절차서 샘플
- 시험 결과보고서 샘플

기기(장비 · 공구)

- 컴퓨터, 윈도우즈 XP 이상
- 사무용 프로그램 (MS Office)
- 프린터
- 인터넷

안전 · 유의 사항

- 공개 소프트웨어를 다운로드하여 사용할 경우 라이선스를 반드시 확인하여 사용 여부를 결정하여야 한다. 일부 소프트웨어는 프리웨어였다가 셰어웨어로 혹은 유료 판매 정책으로 수정할 수도 있다.
- 학습을 위해 다운로드한 프로그램은 학습이 완료된 후 학습장의 지침을 확인한 후 필요 시 프로그램을 제거해야 한다.

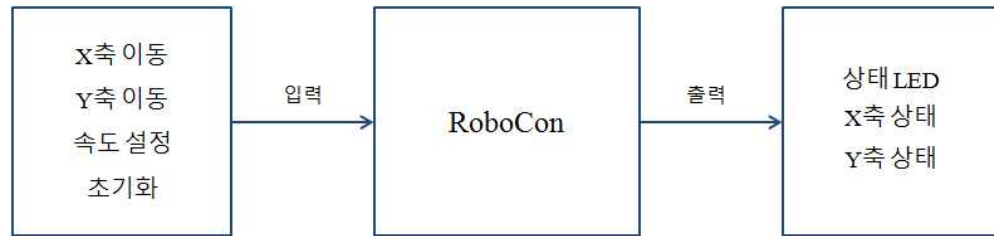
수행 순서

① 시험 대상 프로그램의 예제를 작성한다.

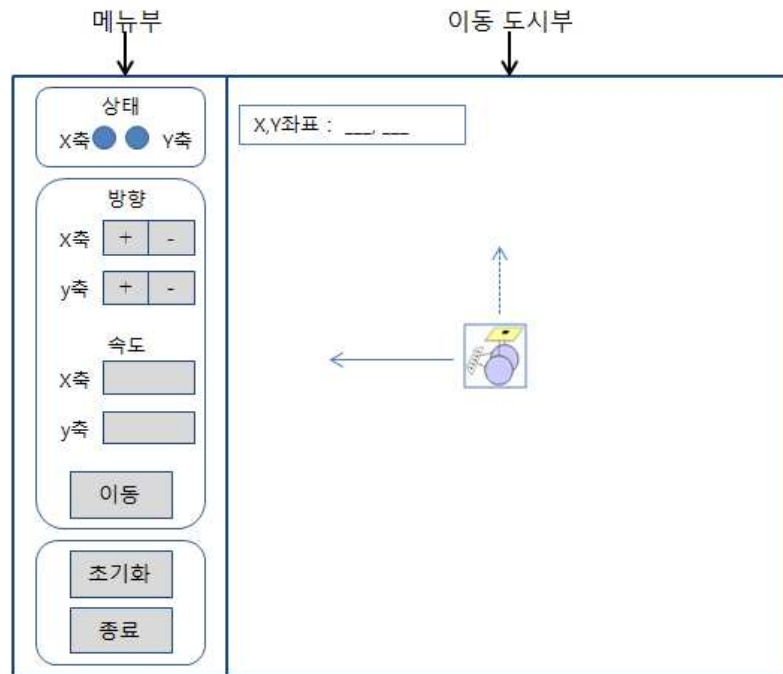
단위 시험을 수행하기 위해서는 시험 대상 프로그램이 반드시 있어야 한다. 간단한 기능을 가진 샘플 프로그램을 작성하고 이를 가지고 단위 시험을 수행한다.

1. 프로그램 개요

- 프로그램명: 프로그램 이름은 프로그래머 임의로 명명한다. 여기서는 RoboCon으로 명명한다.
- 운영 환경: 학습 시 제공된 컴퓨터 운영체제에서 구동될 수 있도록 한다.
- 프로그램 언어: C 혹은 C++
- 프로그램 개요: 로봇의 움직임을 묘사하는 프로그램
- 프로그램 기능: 사용자로부터 명령을 입력받아 이동을 묘사하고 상태값을 출력한다.



[그림1-8] 작성 프로그램 블록도



[그림1-9] 작성 프로그램 화면 예시

위 프로그램의 화면은 예시이며, 각 프로그래머의 취향에 따라 위치, 표현 방법, 제어 방법 등을 다르게 구현한다.

2. 요구 사항

이 프로그램의 요구 조건은 아래와 같다.

- 사용자는 로봇의 현재 위치를(X, Y축) 알 수 있어야 하며 각 축은 -110~110 사이의 값을 가진다.
- 사용자는 X축으로의 이동에 대해 ‘+’ 혹은 ‘-’ 방향으로 제어가 가능해야 한다.
- 사용자는 Y축으로의 이동에 대해 ‘+’ 혹은 ‘-’ 방향으로 제어가 가능해야 한다.
- 사용자는 로봇의 상태를 알 수 있어야 한다. 로봇의 각 축의 위치가 -80 이상이거나 80 이하일 경우는 초록색 LED가, -80보다 작고 -100 이상이거나 80보다 크고 100 이하일 경우는 노란색 LED가, -100보다 작거나 100보다 클 경우는 빨간색 LED가 켜져야 한다.
- 사용자는 이동 속도를 1~10/sec까지 1/sec 단위로 설정할 수 있어야 한다.
- 사용자는 모든 상태를 초기화할 수 있어야 하며, 초기화 시 X, Y축의 상태는 0이어야 한다.

3. 프로그램을 설계한다.

프로그램을 코딩하기 전에 프로그램의 개요와 요구 사항을 바탕으로 프로그램을 설계한다. 프로그램의 설계란 요구 조건을 구현하기 위해 어떤 클래스와 함수를 정의하고 어떤 로직을 적용할 것인지에 대하여 밑그림을 그리는 것으로 산출물은 각종 다이어그램의 형태로 나타난다.

(1) 요구 사항 분석

- 프로그램을 설계하기 전에 먼저 요구 조건을 명확히 파악한다.
- 이 요구 조건을 분석하여 세부 요구 사항을 정의하고 식별자를 부여한다.

<표 1-9> 요구 사항 분석 샘플

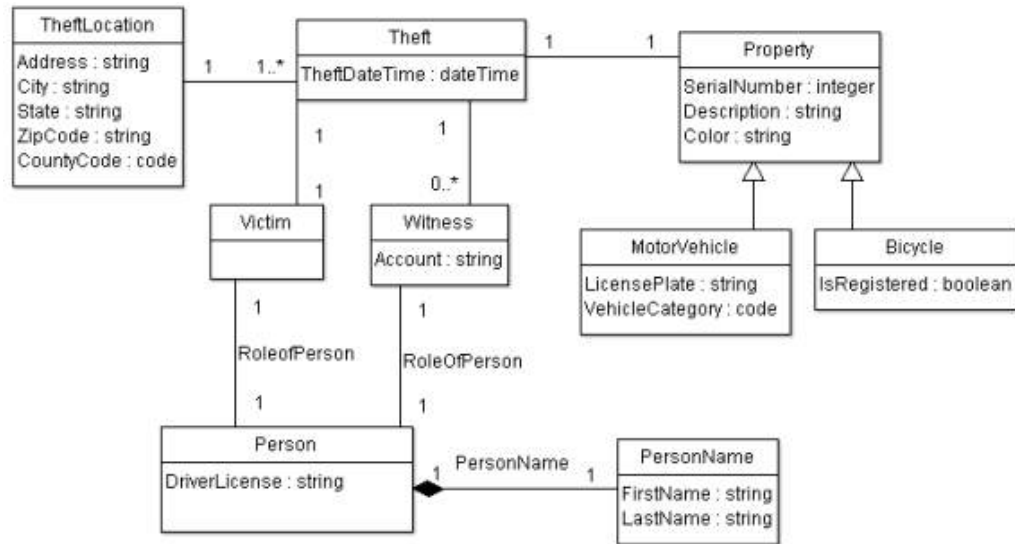
ID	요구 조건	요구 사항 분석
R-01	사용자는 X, Y축의 현재 상태값을 알 수 있어야 하며 각 축은 -110~110사이의 값을 가진다.	로봇은 X, Y 2축을 가지고 있어야 한다.
		각 축 최대값은 110, 최소값은 -110 이다.
R-02		로봇은 현재 축값을 도시해야 한다.
:	:	

(2) 클래스 다이어그램을 작성한다.

분석된 요구 사항을 바탕으로 필요한 다이어그램을 작성한다. 다이어그램을 작성할 땐 전용 유료 툴을 사용할 수도 있고, StarUML이라는 무료 툴을 다운로드하여 사용할 수도 있다(<http://staruml.io/>). 또 파워포인트나 아래한글 등 사무용 프로그램으로 사용해도 무방하다.

여러 다이어그램 중 가장 중요한 핵심이 되는 것이 클래스 다이어그램이다. 클래스 다이어그램에는 클래스명과 함수명 그리고 클래스 간의 연관 관계 등을 명시하여야 한다. Visio, 파워포인트, StarUML 등 다이어그램을 표현할 수 있는 프로그램을 실행시킨다.

- 분석한 세부 요구 사항을 바탕으로 어떤 클래스를 만들지 인터페이스는 어떻게 할지 전체적인 구도를 미리 생각한다.
- 클래스를 표현할 사각형을 그리고 클래스 명을 입력한다.
- 클래스의 속성(attribute)과 연산(operation)을 입력한다.
- 각 클래스 간의 연관 관계를 정의한다.



출처 : <http://www.ibm.com/developerworks/library/x-NIEM1/> 2016. 08. 03 스크린샷.
 [그림 1-10] 클래스 다이어그램 샘플

(3) 기타 다이어그램을 작성한다.

UML에서 정의하고 있는 다이어그램은 클래스 다이어그램 외에도 컴포넌트 다이어그램, 사용 사례 다이어그램(usecase diagram), 순차 다이어그램(sequence diagram), 액티비티 다이어그램(activity diagram) 같은 다양한 다이어그램들이 있다. 그러나 모든 다이어그램을 작성하기보다 프로그램 구현 시 필요하다고 생각되는 다이어그램만 작성해도 된다.

여기서는 제작하는 프로그램의 기능이 복잡하지 않으니 순차 다이어그램 정도만 더 작성하면 충분할 것이다.

4. 프로그램을 코딩한다.

프로그램을 코딩하기 위해서는 프로그래밍 언어와 컴파일 도구를 선택해야 한다. 여기서는 프로그래밍 언어는 ‘C’ 혹은 ‘C++’ 로 선택하고 공개 소프트웨어 중 가볍고 설치도 간편한 ‘DEV-C++’ 이란 컴파일러를 사용한다.

(1) 컴파일러를 다운로드한다.

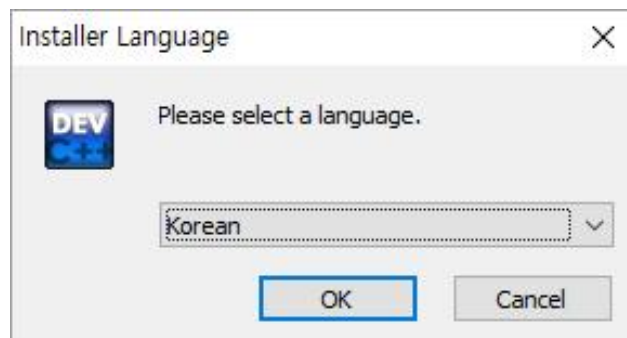
‘DEV-C++’ 이란 프로그램을 다운로드하기 위해서는 웹사이트 <https://sourceforge.net/> 에 접속하여 ‘DEV-C++’ 로 검색하거나 아래 주소를 직접 입력한다.

<https://sourceforge.net/projects/orwelldevcpp/?source=directory>



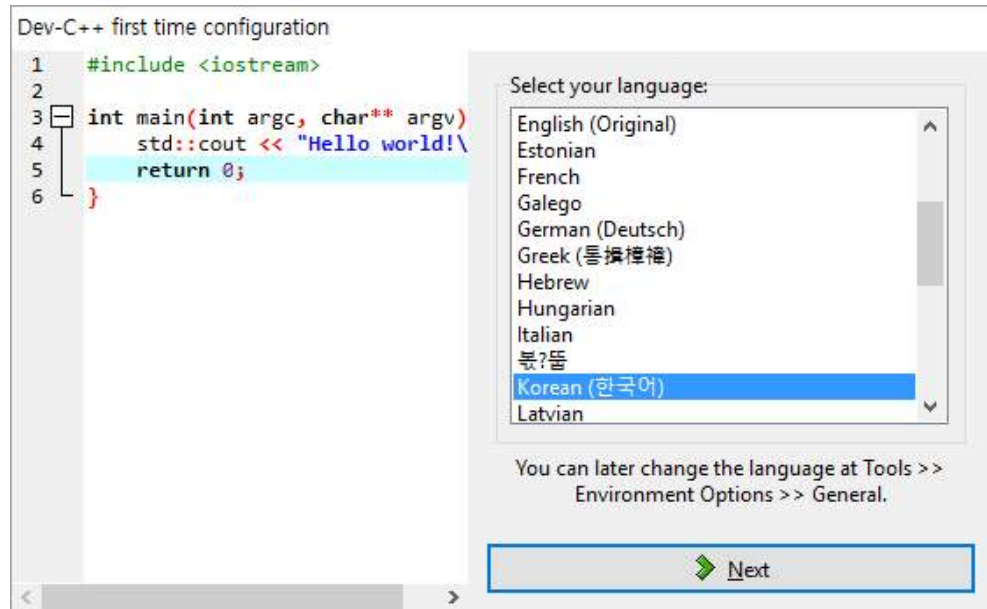
출처 : <https://sourceforge.net/projects/orwelldevcpp/?source=directory> 2016. 08. 02. 스크린샷.
[그림 1-11] DEV-C++ 다운로드 화면

위 웹사이트의 중간에 위치한 ‘다운로드’ 버튼을 눌러 프로그램을 다운로드하고 설치한다. 설치 시 언어는 한국어로 선택한다.



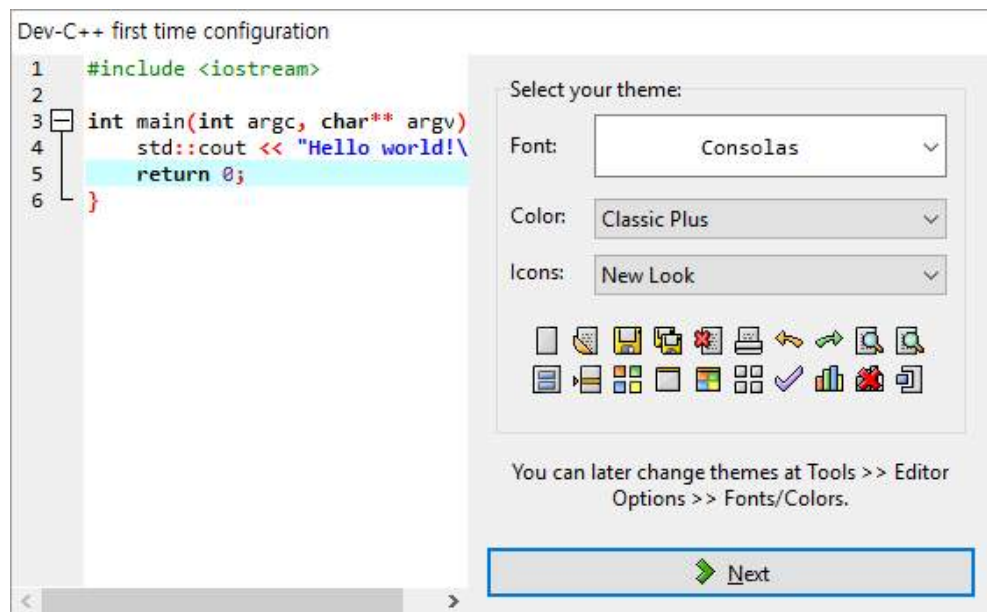
[그림 1-12] 설치 언어 선택

설치가 완료되면 아래와 같은 configuration 화면이 나타난다.



[그림 1-13] Configuration 화면 - 언어

DEV-C++은 한글을 지원하므로 스크롤을 아래로 내려 한글을 선택하고 Next를 누른다.



[그림 1-14] Configuration 화면 - 선택

‘Font’ ‘Color’ ‘Icons’를 원하는 사양으로 선택하고 다음으로 Next를 누른다.

② 시험 절차를 작성한다.

프로그램이 완성되면 프로그램 소스 파일과 설계 자료를 옆의 동료와 서로 교환한다. 교환한 그 프로그램을 시험 대상 프로그램으로 설정하고 시험 절차를 작성한다.

1. 시험 대상 프로그램을 분석한다.

설계 자료를 검토하고 소스 파일을 보면서 함수 목록을 파악하여 기록한다.

<표 1-10> 함수 목록 (예)

구분	번호	함수 명	설명
모듈 / 클래스명	1	AAA	~ 한다.
	2	BBB	~ 한다.
	3	CCC	~ 한다.
	:	:	:

2. 위 정보를 바탕으로 시험 절차를 작성한다.

<표 1-11> RoboCon 프로그램 단위 시험 절차서 (예시)

1. 개요
 - 문서명 : RoboCon 단위 시험 절차서_V0.1
 - 프로그램명 : RoboCon
 - 프로그램 설명 : Robot 이동 제어 시뮬레이션 프로그램
2. 참고 자료
 - RoboCon 시험 계획서
3. 시험 준비 사항
 - 시험 형상
 - 소프트웨어 및 하드웨어 준비 사항
4. 시험 절차
 - 시험 목록
 - 테스트 케이스
5. 추적성
 - 요구 사항 추적성 매트릭스

3. 테스트 케이스를 작성한다.

테스트 케이스는 요구 사항과 프로그램 소스 코드 및 설계 산출물로부터 추출해서 작성해야 하며 아래와 같은 항목들이 들어간다. 테스트 케이스는 반드시 많다고 좋은 것은 아니나 요구 사항을 모두 확인할 수 있을 만큼 정의해야 한다.

- 프로그램명: 테스트를 수행할 프로그램 이름
- Test Case ID: 테스트 케이스의 식별자
- Req ID: 요구 사항 추적성 ID
- 모듈 이름: 테스트를 수행할 모듈의 이름
- Test Case 설명: 수행할 테스트에 대한 설명
- 테스트 조건: 테스트를 실행하기 위한 전제 조건, 예를 들어 ‘로그인’ 테스트를 하기 위해서는 ‘사용자는 계정을 가지고 있어야 한다.’를 정의해야 함.
- 입력값: 테스트 실행에 필요한 입력값
- 예상 출력값: 테스트 결과 출력되리라 예상되는 값
- 실행 결과: 테스트 실행 결과 출력되는 실제 값
- pass or fail: 테스트 실행 결과 합격/불합격 상태
- 작성자: 테스트 케이스 작성자
- 시험자/시험일: 테스트 케이스 시험자/시험일

<표 1-12> 샘플 프로그램용 테스트 케이스 (예시)

프로그램명 : RoboCon			단계 : 단위 시험		
테스트 케이스					
Test Case ID : TC-1-001			Req ID : R-01		
모듈 이름 : Move_X0			작성자 : 홍XX		
시험자 : 김XX			시험일 : 201X. XX. XX		
Test Case 설명 : x축으로 이동하는 모듈의 기능 점검					
테스트 조건 : -					
단계	입력값	예상 출력값	실행 결과	Pass/Fail	조치 사항
1	XX	XXX			
2	XX	XXX			
3	:	:			
4					

4. 요구 사항 추적성 매트릭스를 작성한다.

- ① 요구 사항 ID: 요구 사항의 식별자로 추후 추적성 관리에 사용됨
- ② 요구 사항 내용: 요구 사항 내용
- ③ Test Case ID: 테스트 케이스 작성 시 정의한 ID, 추후 보고서 작성이나 추적성 관리 등에 사용됨
- ④ 시험명: 테스트 케이스 작성 시 정의한 시험명

시험 방법이나 시험자 등 추가 사항을 표에 삽입하여 향후 관리에 사용할 수도 있다.

<표 1-13> 요구 사항 추적성 매트릭스 (예시)

①Req ID	②요구 사항 내용	③Test Case ID	④시험명
R-01	사용자는 X, Y축의 현재 상태값을 알 수 있어야 하며 각 축은 -110 ~ 110 사이의 값을 가진다.	TC-1-001	X축 이동
		TC-1-002	Y축 이동
		TC-1-003	:
		:	:
R-02	사용자는 X축으로의 이동에 대해 '+' 혹은 '-' 방향으로 제어가 가능해야 한다.	TC-1-00X	:
		TC-1-00X	:
		:	:
:	:	:	:

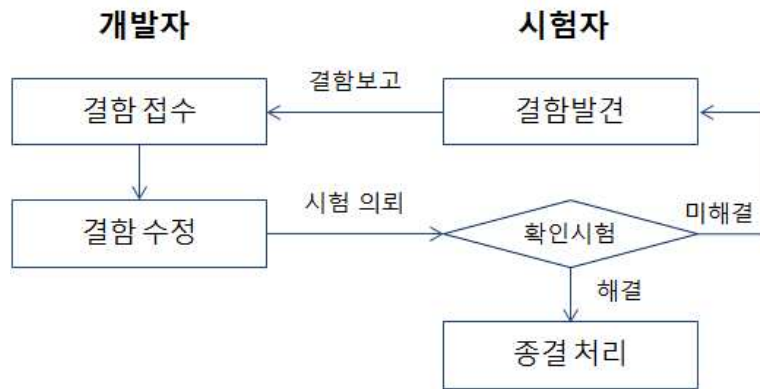
③ 단위 시험을 수행한다.

1. 시험자는 테스트 케이스 별로 시험을 수행한다.

각 테스트 케이스 별 시험을 수행 시 테스트 코드를 구현하여 시험한다. 자동화 도구가 있으면 도구에서 자동으로 해주는 경우가 있다. 이에 대해서는 ‘학습 내용 3-2의 필요 지식’ 을 참조한다.

2. 결함을 리포팅한다.

시험을 수행 중 결함이 발생하면 아래와 같은 결함 처리 프로세스를 따른다. 결함이 발생하였을 때는 가능하면 재현 테스트를 할 수 있도록 결함의 발생순서나 상황을 가급적 자세하게 기록한다.



[그림 1-17] 결함 처리 프로세스

④ 시험 결과보고서를 작성한다.

1. 작성 내용

시험 결과보고서에는 시험의 대상, 시험 환경, 시험 항목, 시험의 절차와 방법을 간단히 작성한 후 시험 결과를 정리하여야 한다.

2. 작성 예시를 보고 시험 결과보고서를 작성한다.

<표 1-14> 시험 결과보고서 (예시)

1. 개요

- 문서명 : RoboCon 단위 시험 결과보고서
- 프로그램명 : RoboCon
- 프로그램 설명 : Robot 이동 제어 시뮬레이션 프로그램

2. 참고 자료

- 요구 사항 목록
- RoboCon 시험 절차서

3. 시험 결과

- 시험 목록 및 결과 (요구 사항 추적성 매트릭스 포함)
- 결함 목록 및 처리 결과

4. 결론

수행 tip

- 테스트 케이스를 작성할 시 입력값에 Positive 경우와 Negative 경우 모두 설정하여 확인한다.
- Code Coverage에 대한 개념을 학습한다.

교수 방법

- C++ 언어와 DEV-C++ 컴파일 도구 개발환경에 익숙해지도록 독려한다.
- 예제 프로그램을 코딩할 시 3개 이상의 클래스로 구성될 수 있도록 하고 의도적으로 논리적인 결함을 내포해서 만들도록 하여 시험자가 결함을 찾아내도록 유도한다.
- 시험용 프로그램은 ‘학습모듈 2. 통합기능 시험평가하기’와 ‘학습모듈 3. 신뢰성 시험평가하기’에서도 사용되니 만일, 코딩 능력이 조금 부족한 학습자가 있다면 직접 지도하거나 주변의 학습자에게 도움을 요청해서라도 주어진 요구 사항에 반드시 부합되게 완성하도록 유도한다.
- 요구 사항을 추가하거나 수정해서 학습자들이 프로그램의 구성과 클래스 등의 설계를 다양하고 창의적으로 할 수 있도록 한다.
- 시험 절차서와 결과보고서를 작성 시 문서의 구성과 제목 정도가 들어가 있는 양식을 제공하여 참조해서 작성할 수 있도록 한다.
- 1~2개 정도 테스트 케이스를 작성하는 법에 대해 시범을 보인다.
- 학습자 각자에게 1~2개 항목에 대해 테스트 케이스를 작성하도록 하고 설명하도록 하고 나머지 학습자들과 상호 토론을 하도록 유도한다. 이를 통해 테스트 케이스를 적절하고 효과적으로 작성하는 방법을 익히게 한다.
- 시험 시 발생하는 이슈 항목이나 중요 결과에 기록을 습관화시킬 수 있도록 강조한다.

학습 방법

- C++ 언어의 프로그래밍 하는 방법과 DEV-C++ 컴파일 도구 개발환경에 익숙하도록 사용법을 사전에 연습한다.
- 예제 프로그램 코딩 시에는 3개 이상의 클래스로 구성하도록 하며 프로그램은 수행이 되나 단위 시험으로 찾아낼 수 있도록 논리적인 결함 등을 임의로 추가한다.
- 예제 프로그램은 ‘학습모듈 2. 통합기능 시험평가하기’와 ‘학습모듈 3. 신뢰성 시험평가하기’에서도 사용되니 프로그램에 대해 이해하고 완성해야 한다.

- 요구 사항이 추가되거나 수정되면 그에 맞도록 클래스나 프로그램 로직을 재설계해야 한다.
- 시험 절차서 및 시험 결과보고서를 용이하게 작성하기 위해 문서 작성 프로그램을 연습한다.
- 시험을 계획할 시 테스트 케이스 작성이 가장 중요한 사항 중 하나이니 작성하는 법에 대해서 충분히 연습해야 한다.
- 테스트 케이스 작성해보고 다른 학습자들이 작성한 방법과 비교하고 상호 토론하여 테스트 케이스를 다양하게 작성할 수 있음을 확인한다.
- 시험 시 발생하는 이슈 항목이나 중요 결과에 대해서는 당시의 기록이 중요하다. 시험 중에는 늘 기록하는 습관을 가져야 한다.

학습1 평 가

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	평가 항목	성취수준		
		상	중	하
단위기능 시험평가	- 단위기능의 정상적 수행 여부를 판단할 수 있는 시험절차서를 작성할 수 있다.			
	- 시험절차서에 따라 단위기능을 구현한 함수의 동작이 적절한지 판단할 수 있다.			
	- 단위기능을 시험한 결과를 보고서로 작성할 수 있어야 한다.			

평가 방법

- 포트폴리오

학습 내용	평가 항목	성취수준		
		상	중	하
단위기능 시험평가	- 시험 대상 프로그램			
	- 단위기능 시험 절차서			
	- 테스트 케이스			
	- 결함 리포트			
	- 시험 결과보고서			

- 문제 해결 시나리오

학습 내용	평가 항목	성취수준		
		상	중	하
단위기능 시험평가	- 제시된 요구 사항을 분석하여 설계에 적절하게 반영			
	- 설계 문서대로 코딩			
	- 요구 사항과 설계 문서로 테스트 케이스 작성			
	- 주어진 테스트 케이스로 테스트 코드 만들어 테스트			

• 서술형 시험

학습 내용	평가 항목	성취수준		
		상	중	하
단위기능 시험평가	- 화이트박스 시험과 블랙박스 시험에 대한 구분			
	- 요구 조건을 분석하여 하위 요구 사항 도출			
	- 제시된 요구 사항을 분석하여 설계 문서 만들기			
	- 설계 문서로 테스트 케이스 작성			

• 평가자 질문

학습 내용	평가 항목	성취수준		
		상	중	하
단위기능 시험평가	- 화이트박스 시험과 블랙박스 시험에 대한 구분			
	- 요구 조건을 분석하여 하위 요구 사항 도출			
	- 제시된 요구 사항을 분석하여 설계 문서 만들기			
	- 설계 문서로 테스트 케이스 작성			

• 일지/저널

학습 내용	평가 항목	성취수준		
		상	중	하
단위기능 시험평가	- 테스트 케이스 작성 검토 후 수정 반영 여부			
	- 결함 발견 시 처리 과정			
	- 절차서 작성, 시험평가 과정의 기록			

• 작업장 평가

학습 내용	평가 항목	성취수준		
		상	중	하
단위기능 시험평가	- 요구 사항 분석 → 설계 → 코딩 → 시험 과정 준수 여부			
	- 시험평가 과정 및 결함 발견 시 기록 여부			

피드백

1. 포트폴리오

- 작성된 테스트 케이스를 검토하여 요구 사항을 반영하지 못한 부분에 대해 설명하고 보완해서 시험을 실시하도록 한다.

2. 문제 해결 시나리오

- 요구 사항으로부터 테스트 케이스를 도출하는 과정과 테스트 케이스를 만드는 방법에 대해 의견을 듣고 예시를 제시한다.

3. 서술형 시험

- 각 학습자가 제출한 시험의 결과를 공유하여 요구 사항을 도출하는 다양한 방법, 테스트 케이스를 만드는 다양한 방법 등을 경험하게 한다.

4. 평가자 질문

- 화이트박스, 블랙박스 시험에 대한 개념에 대한 질문 목록과 상위 요구 조건에서 하위 요구 조건을 도출해내기 위한 과제를 준비한다.

5. 일지/저널

- 시험 결과보고서 및 결함이 생겼을 때 조치한 결과물 등을 제출하도록 하고 검토 후 필요 없는 항목이 기록되었거나 중요한 항목이 누락되었을 경우 첨삭하여 회신해 준다.

6. 작업장 평가

- 시험 과정에서 이슈 사항 발생 시 혹은 결함 발견 시 기록하는 습관에 대해 강조한다.

학습 1	단위기능 시험평가하기
학습 2	통합기능 시험평가하기
학습 3	신뢰성 시험평가하기

2-1. 통합기능 시험평가

학습 목표

- 통합기능의 정상적 수행 여부를 판단할 수 있는 시험 절차를 작성할 수 있다.
- 통합 시험 절차에 따라 단위 소프트웨어와 소프트웨어 컴포넌트를 통합한 프로그램의 운용로직 구현 여부를 평가할 수 있다.
- 통합기능 시험평가 결과를 보고서로 작성할 수 있다.

필요 지식 /

① 통합 시험

1. 통합 시험의 목적 및 정의

통합 시험이란 각 단위 모듈들을 시험한 다음 단계의 시험으로 모듈들이 기능적으로 모여 시스템이나 서브시스템을 형성하게 되는데, 이 과정에서 발생할 수 있는 문제점을 검증하는 것이다.

단위 시험 시에는 시험을 위해 실제 사용되는 코드가 아닌 테스트 드라이버와 스텝을 사용하였으나 이는 가상의 임시적인 것이므로, 모듈을 통합하였을 경우 모듈 간의 상호작용이 제대로 이루어지는지 검사하는 과정이다.

2. 통합 시험 절차

통합 시험을 위해서는 단위 시험 때와 마찬가지로 테스트 진행을 위해 필요한 추가 테스트 케이스를 작성해야 한다. 요구 사항과 설계 문서를 검토하여 코드 분석을 하고 이를 통해 사용되는 변수의 입/출력 정보나 필요한 스텝 정보를 반영하여 작성한다.

통합 시험에서는 시험의 대상 범위와 방법을 설정하게 되는데, 대상 범위는 작게는 모듈 단위부터 크게는 태스크 단위로 묶어서 진행할 수 있다.

이렇게 요구 사항 및 코드 분석 그리고 시험의 대상과 방법을 설정한 이후 이 정보를 바탕으로 테스트 케이스를 작성하고 절차화하고 절차대로 통합 시험을 수행한다.

② 통합 시험 절차서

1. 개요

테스트 목적과 일정, 조직 구성 등을 정의한다.

2. 테스트 케이스

테스트 케이스를 정의한다. 테스트 케이스는 각 시험 대상 소프트웨어의 규모, 성격 그리고 복잡도 등에 따라 작성하는 범위나 내용이 달라질 수 있다. 간단하게 요구 사항 ID, 테스트 케이스 ID, 입출력 데이터 값 정도로 정리할 수도 있고, 각 테스트 케이스마다 테스트용 스크립트까지 작성할 수도 있다.

3. 시험 절차서 샘플 예제

(1) 개요

<표 2-1> 시험 절차서 샘플 - 개요

예제)

1. 개요

1.1 소프트웨어 개요

XXX 소프트웨어는 OO시스템의 각 단계별 점검 및 총조립 점검을 하고 단계별 인터페이스를 모사해 주는 시스템이다.

[그림 X.] 소프트웨어 구성도

1.2 목적

본 SW 시험 절차서는 XXX 소프트웨어의 연동 시험에 대한 개요, 참고 문서, 시험 준비 사항, 시험 내역, 요구 사항 추적성, 참고 사항 등을 규정한 문서이다.

1.3 시험 식별자

기능	기능 구분	비고
소프트웨어 기능	SFR	software functional requirement
인터페이스	IFR	interface requirement
안전 및 보안	SSR	safety & security requirement
환경 및 컴퓨터 자원	ECR	environment & computer resource requirement

(2) 참고 문서: 시험 절차를 작성하는 데 참고로 사용된 문서

<표 2-2> 시험 절차서 샘플 - 참고 문서

예제)

2. 참고 문서

- 소프트웨어 요구 사항 명세서(SRS, software requirement specification)
- 소프트웨어 설계 기술서(SDD, software design description)
- MIL-STD-498 software development and documentation

문서 번호	문서명
XXX-SRS-XX	XXX 소프트웨어 요구 사항 명세서(SRS, software requirement specification)
XXX-SDD-XX	XXX 소프트웨어 설계 기술서 (SDD, software design description)
XXX-XXX-XX	XXX 소프트웨어 규격서
MIL-STD-498	software development and documentation

(3) 시험 항목

<표 2-3> 시험 절차서 샘플 - 시험 항목

예제)

3. 시험 항목

요구 사항 식별자	요구 사항	시험 식별자	시험명
R-SFR-00X	AAA HW/SW 상태 감시	TC-SFR-00X TC-SFR-00X	AAA 시스템 기동 AAA 시스템 재기동
R-SFR-00X	BBB HW/SW 상태 감시	TC-SFR-00X TC-SFR-00X	BBB 시스템 기동 BBB 시스템 재기동
R-SFR-00X	로그 관리	TC-SFR-00X	로그 관리
R-SFR-00X	경고 및 알림	TC-SFR-00X	텍스트 경고 출력
		TC-SFR-00X	음성 경고 출력
:	:	:	:
R-SFR-0XX	Network 상태 감시	TC-SFR-0XX	네트워크 상태 감시

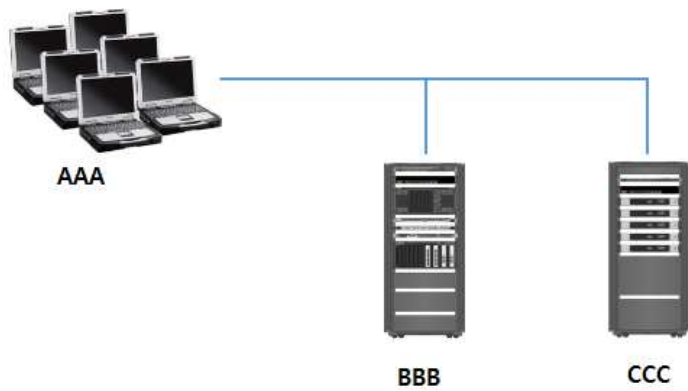
(4) 시험 준비 사항

<표 2-4> 시험 절차서 샘플 - 시험 준비 사항

예제)

4. 시험 준비 사항

4.1 시험 형상



[그림 X.X] XXX 시스템 통합 시험 형상

4.2 하드웨어 준비 사항

구성품	사양	수량	용도
AAA 컴퓨터	Intel i7 이상, RAM 8GB 이상	2ea	AAA SW 탑재용 본장비
BBB	Intel i7 이상, RAM 8GB 이상	1ea	계통별 모니터링
CCC	MVME7100	1ea	제어기 기능 확인
기타	전원 케이블, USB 메모리		

(5) 시험 절차 및 테스트 케이스

<표 2-5> 시험 절차서 샘플 - 시험 절차

예제)

5. 시험 절차

5.1 AAA 시스템 기동

시험명	AAA 시스템 기동	식별자	TC-SFR-00X
설명	AAA 시스템 기동 시 정상적으로 BBB와 연동되는지 점검한다.		
사전 조건	AAA 시험용 컴퓨터에 방화벽 사용 안함으로 되어 있고, SW형상 적용이 되어있는 상태이다.		
절차	1. [BBB SW] 메인 화면에서 상태 감시제어 > 시스템 감시제어 버튼을 클릭한다. (√) 2. [BBB SW] 시스템 감시 윈도우에서 AAA1 의 상태를 확인한다. (√) 3. [AAA] 시스템을 기동하여 XX SW를 실행한다. (√) 4. [BBB SW] 시스템 감시 윈도우에서 XX-AAA1 네트워크 연결 상태를 확인한다. (√) 5. [BBB SW] 시스템 감시 윈도우에서 XX-AAA1 의 상태를 확인한다. (√)		
예상 결과	1. 시스템 감시 윈도우가 전시된다. 2. XX-AAA1 콘솔 아이콘의 테두리(㉠)가 적색으로 전시된다. 3. XX-AAA1의 SW 들이 자동 시작 된다. 4. 네트워크에 연결된 XX-AAA1의 연결선(㉠)이 녹색으로 전시된다. 5. XX-AAA1의 모든 상태(CPU 사용량, 디스크 사용량, 메모리 사용량, 프로세스 상태)가 정상이면 콘솔 아이콘의 테두리(㉠)가 녹색으로 전시된다.		

5.2 AAA 시스템 재기동

시험명	AAA 시스템 재기동	식별자	TC-SFR-00X
설명	AAA 시스템 재기동 시 정상적으로 BBB와 연동되는지 점검한다.		
사전조건	1. BBB 시험용 컴퓨터에 방화벽 사용 안함으로 되어 있고, BBB SW 가 구동중인 상태이다. 2. AAA 시험용 컴퓨터에 방화벽 사용 안함으로 되어 있고, SSI SW가 구동중인 상태이다.		
절차	1. [BBB SW] 메인화면에서 상태 감시제어 > 시스템 감시제어 버튼을 클릭한다. (√) 2. [BBB SW] 시스템 감시 윈도우에서 XX-AAA1 의 상태를 확인한다. (√) 3. [BBB SW] 시스템 감시 윈도우에서 XX-AAA1 네트워크 연결 상태를 확인한다. (√) 4. [BBB SW] XX-AAA1 콘솔 아이콘을 클릭한다. (√) 5. [BBB SW] XX-AAA1 상태 윈도우에서 프로세스 목록 관리(㉠)를 확인한다. (√)		
예상 결과	1. 시스템 감시 윈도우가 전시된다. 2. XX-AAA1의 모든 상태(CPU 사용량, 디스크 사용량, 메모리 사용량, 프로세스 상태)가 정상이면 콘솔 아이콘의 테두리(㉠)가 녹색으로 전시된다. 3. 네트워크에 연결된 XX-AAA1의 연결선(㉠)이 녹색으로 전시된다. 4. XX-AAA1 상태 윈도우가 전시된다. 5. XX-AAA1 시험용 컴퓨터의 SW 목록과 실행 상태가 전시된다.		

5.3 BBB 시스템 기동

:
:
:

(6) 요구 사항 추적성 매트릭스

<표 2-6> 시험 절차서 샘플 - 요구 사항 추적성 매트릭스

예제)

6. 요구 사항 추적성 매트릭스

소프트웨어 요구 사항		검증 방법				비고
ID	내용	검사	분석	데모	시험	
R-SFR-00 X	AAA 시스템을 기동할 수 있어야 한다.			√		
	AAA 시스템을 재기동할 수 있어야 한다.			√		
R-SFR-00 X	BBB 시스템을 기동할 수 있어야 한다.			√		
	BBB 시스템을 재기동할 수 있어야 한다.			√		
:	:					

③ VCRM(verification cross reference matrix)

1. 정의

VCRM은 요구 사항과 그 요구 사항을 검증하는 방법을 매트릭스로 표현한 것으로 보통 시스템 통합 시험 시 시스템의 요구 사항을 어떠한 방법으로 검증할 것인가에 대해 시험 계획 시 정의한다. 이때 만일, 요구 사항 명세서가 정의되어 있다면 소프트웨어 요구 사항을 직접 기술하는 대신 SRS(software requirement specification)의 장 절을 인용해도 된다.

2. 검증 방법

(1) 검사(inspection)

기능 항목, 관련 문서, 프로그램 코드 등 정량적으로 판단하기 어려운 항목에 대해 직접 육안으로 확인하여 요구 사항의 만족 여부를 확인하는 방법이다.

(2) 분석(analysis)

분석이란 모델링이나 시뮬레이션 방법, 계산 등 다른 검사 방법을 사용해서 획득한 자료를 분석하여 요구 사항의 만족 여부를 확인하는 방법이다.

(3) 데모(demonstration)

특정한 시험 도구를 사용하지 않고 기능을 직접 수행하여 요구 사항이 만족되는지를 확인하는 방법이다.

(4) 시험(test)

적절한 시험 조건에서 시험 자료를 이용하여 실제로 기능을 수행시켜서 요구 사항이 만족되는지 확인하는 방법이다.

3. 샘플 예제

<표 2-7> VCRM 샘플

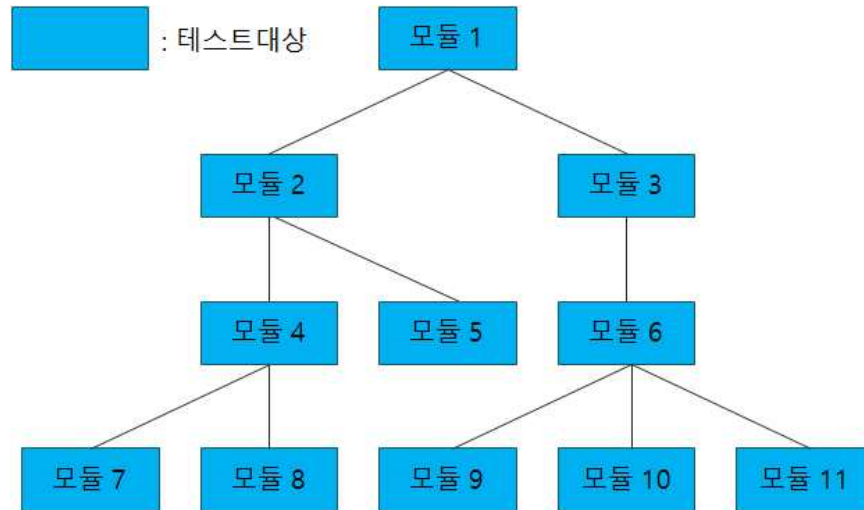
소프트웨어 요구 사항		검증 방법				비고
ID	내용	검사 Inspection	분석 Analysis	데모 Demonstration	시험 Test	
R-01	사용자는 X, Y축의 현재 상태값을 알 수 있어야 하며 각 축은 -110~110사이의 값을 가진다.	V		V		
R-02	사용자는 X축으로의 이동에 대해 '+' 혹은 '-' 방향으로 제어가 가능해야 한다.			V		
	:					
	:					
R-XX ..				V	V	
:	:					:

④ 시험 수행 기법

통합 시험은 모듈을 통합시키는 방법에 따라 여러 가지 테스트 기법이 있다.

1. 빅뱅 기법

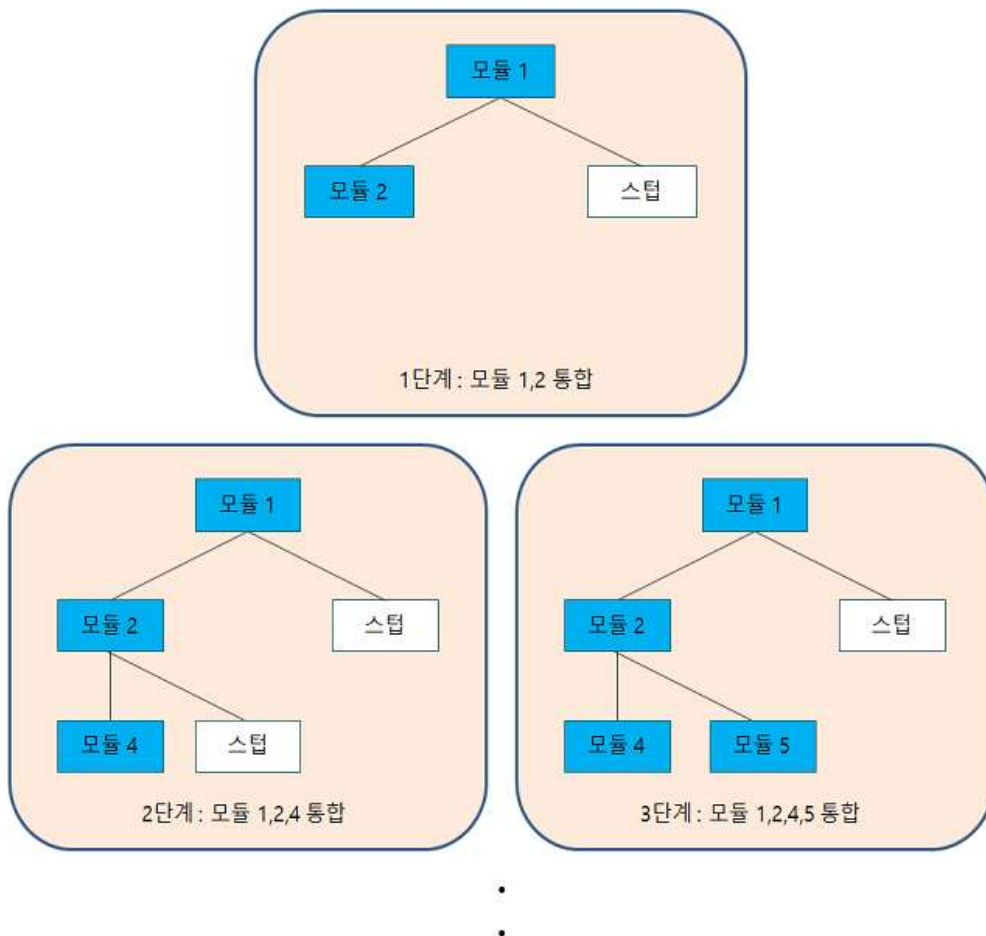
모듈을 한꺼번에 통합하여 테스트하는 기법으로 가장 빨리 시험할 수 있지만 오류가 날 경우 어느 모듈에서 문제가 생겼는지 확인하기 어려운 단점이 있다. 주로 프로그램의 코드가 별로 크지 않는 경우에 사용한다. 시험하는 절차는 [그림 2-1]과 같다.



[그림 2-1] 빅뱅 기법

2. 하향식(top-down) 기법

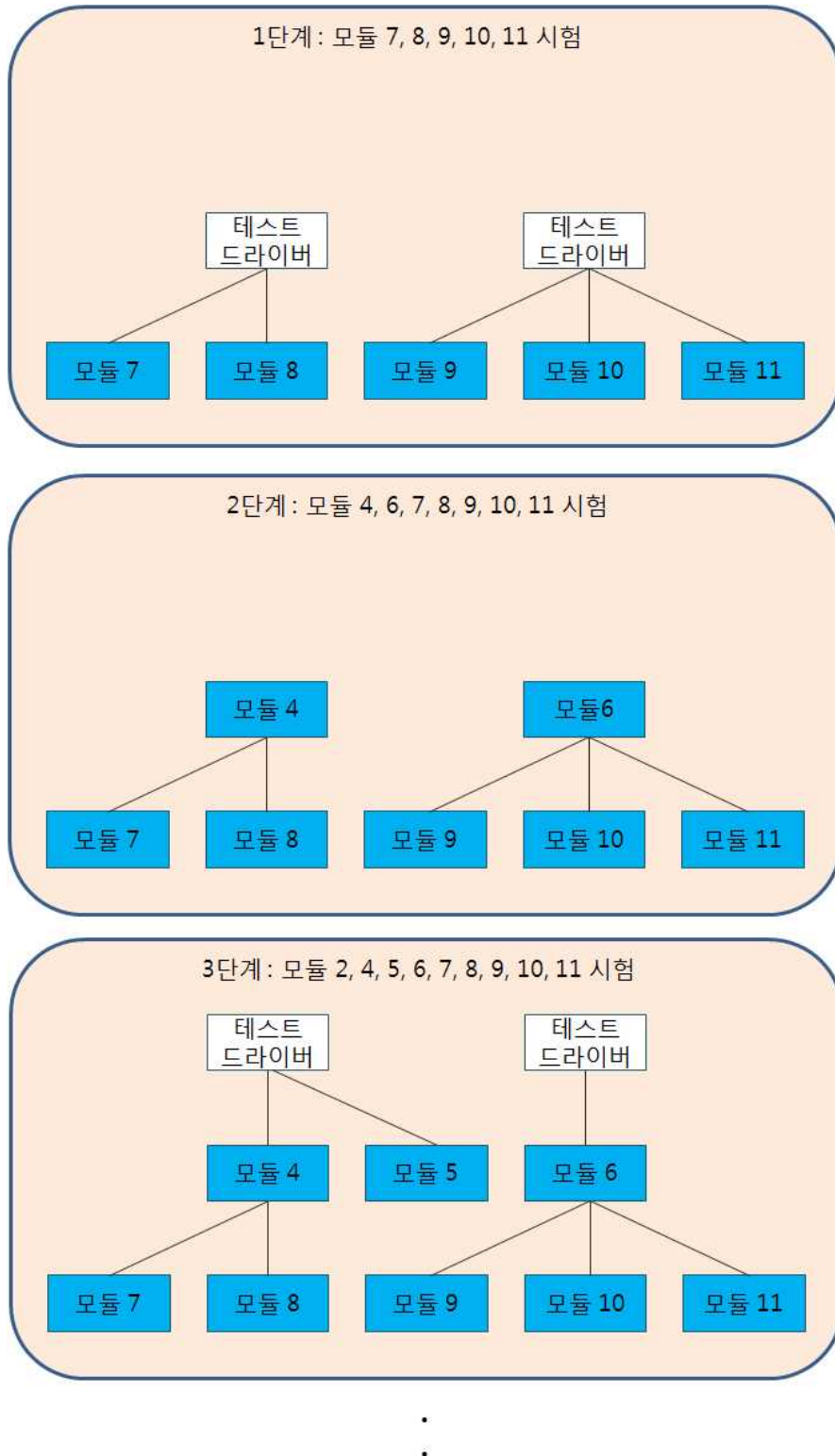
[그림 2-2]와 같이 최상위 모듈부터 하위로 호출되는 모듈을 점진적으로 통합하는 기법으로 상위 모듈 시험 시 하위 모듈에 대한 스텝이 필요하다.



[그림 2-2] 하향식 기법

3. 상향식(bottom-up) 기법

[그림 2-3]와 같이 최하위 모듈부터 상위 모듈로 통합해 가면서 시험하는 방법으로 하위 모듈 시험 시 상위 모듈에 대한 테스트 드라이버가 필요하다.



[그림 2-3] 상향식 기법

⑤ 시험 방법

1. 블랙박스 시험

블랙박스 시험은 내부 구조에 대한 로직보다는 구현되는 기능이나 성능만을 시험한다.

(1) 신택스 시험(syntax testing)

신택스 시험은 입력 데이터가 미리 정의된 조건에 부합하는지 확인하는 기법이다. 예를 들어 제어 속도를 입력받아 처리하는 프로세스의 테스트 케이스를 생각해 보자.

- 입력 변수: 제어 속도
- valid 조건: 0~10
- invalid 조건: 음의 숫자, 10보다 큰 숫자, 문자, 특수문자, 소수점 등

이때 신택스 시험 기법을 적용하면 테스트 케이스는 아래 표와 같이 정의할 수 있다.

<표 2-8> 신택스 시험 기법

순번	입력 유형	입력 값	예상 결과
1	0~10 사이 숫자	5	정상 처리
2	10보다 큰 숫자	180	에러 메시지
3	특수문자	@\$	에러 메시지
4	소수점	3.9	에러 메시지

(2) 등가 분할(equivalence partitioning)

등가 분할은 입력값의 범위를 유사한 부분집합으로 나누고, 그 값에 대해 시험하는 방법이다. 시험 조건을 만드는 것이 간단하고 무작위로 샘플링하는 방법보다는 체계적이라 할 수 있다.

로봇의 위치에 따라 알람을 표시해 주는 프로그램의 테스트 케이스는 아래와 같다.

- $-80 \leq \text{위치} \leq 80$: 초록색(G)
- $-100 \leq \text{위치} < -80$ or $80 < \text{위치} \leq 100$: 노란색(Y)
- $-100 > \text{위치}$ or $100 < \text{위치}$: 빨강색(R)

<표 2-9> 등가 분할 시험 기법

이동속도	-80 ≤ 위치 ≤ 80	-100 ≤ 위치 < -80 80 < 위치 ≤ 100	-100 > 위치 100 < 위치
입력값	25	-89	109
예상 결과값	초록색(G)	노란색(Y)	빨강색(R)
실제 결과값			

(3) 경계값 분석(boundary value analysis)

경계값 분석은 등가 분할과 비슷하나 부분집합의 경계값이나 주위값을 테스트 케이스로 추출하는 방법이다. 위 예를 적용하여 경계값 분석 기법으로 테스트 케이스를 만들면 아래와 같다.

<표 2-10> 경계값 분석 기법

시험 케이스	1	2	3	4	5	6	7	8	9	10	11	12
입력값	-101	-100	-99	-81	-80	-79	79	80	81	99	100	101
예상 결과값	R	Y	Y	Y	G	G	G	G	Y	Y	Y	R
실제 결과값												

(4) 기타

그 외에도 상태전이 테스트, 유즈케이스 테스트 등 다양한 기법들이 있다.

⑥ 통합 시험 결과보고서

시험 결과보고서도 시험 절차서나 테스트 케이스를 만드는 것과 마찬가지로 각 프로젝트나 소프트웨어마다 다를 것이다. 중요한 것은 보고서의 목적에 부합되도록 작성하기만 하면 형식에는 크게 구애를 받지 않아도 된다.

시험 결과보고서의 목적은 시험의 결과를 정리하고 이 결과를 바탕으로 소프트웨어의 품질에 대한 평가와 권고 사항 등을 제공하는 것이다.

시험 결과보고서는 반드시 각 시험 단계마다 나올 필요는 없다. 작은 프로젝트 같은 경우는 단위 시험, 통합 시험, 시스템 시험까지 합쳐서 1개의 보고서로 나올 수도 있고, 기간이 길고 규모가 큰 프로젝트의 경우는 매 시험마다 결과보고서를 요청할 수도 있다. 또 단위 시험 결과보고서는 아래 표와 같이 합격/불합격의 결과만 간단하게 정리할 수도 있다.

<표 2-11> 간략한 단위 시험 결과보고서 사례

시험식별자	시험명	합격	불합격	비고
TC-SFR-001	AAA 시스템 기동	√		
TC-SFR-002	AAA 시스템 재기동	√		
TC-SFR-003	BBB 시스템 기동	√		
TC-SFR-004	BBB 시스템 재기동		√	
TC-SFR-005	AAA 시스템 상태 감시	√		
TC-SFR-006	로그 관리	√		
TC-SFR-007	화면 캡처	√		
:	:	√		
:	:			
합 계		00	0	0

full 버전의 정식 시험 결과보고서에는 아래와 같은 항목들로 구성되는 것이 일반적이다.

<표 2-12> 시험 결과보고서 사례

XX 소프트웨어 시험 결과보고서	
1. 개요	
1.1 문서 번호	
1.2 적용 범위	
1.3 참고 문서	
2. 시험 결과	
2.1 종합	
2.2 세부시험 결과	
2.3 이슈 사항	
2.4 결론 및 권고사항	
3. 참고	
3.1 용어 정리	
3.2 문서 변경 절차 및 재개정 이력	

수행 내용 1 / 통합기능 시험 절차서 작성하기

재료 · 자료

- 노트 및 필기도구
- 소프트웨어 요구 사항 명세서
- 기타 소프트웨어 설계 산출물 (소프트웨어 설계 명세서, 클래스 다이어그램 등)
- 시험 대상 프로그램(RoboCon) 소스 코드

기기(장비 · 공구)

- 컴퓨터, 윈도우즈 XP 이상
- 사무용 프로그램 (아래한글 or MS Office)
- 프린터
- 인터넷

안전 · 유의 사항

- 시험 절차서에는 요구 사항을 검증할 수 있는 모든 테스트 케이스가 정의되어야 한다.
- 절차서 양식은 크게 구매받을 필요가 없으며 가장 중요한 테스트 케이스만 잘 정리되어 있으면 문제가 없다.

수행 순서

① 요구 사항 및 설계 문서를 파악한다.

학습모듈 1-1에서 작성한 RoboCon 프로그램을 이용하여 요구 조건 및 요구 사항 분석 결과 문서와 클래스 다이어그램 등의 설계 문서를 자세히 검토한다.

② 시험 절차서를 작성한다.

1. 개요를 작성한다.

소프트웨어 개요, 시험의 목적 등을 기술한다.

2. 참고 문서를 작성한다.

요구 사항 명세 및 설계 참고 자료를 기술한다.

3. 시험 항목 및 테스트 케이스를 작성한다.

요구 사항 명세 및 설계 참고 자료를 검토한 후 시험 항목과 테스트 케이스를 작성한다.

(1) 시험 항목 식별

시험 항목은 소프트웨어 요구 사항에 나와 있지 않은 항목도 포함될 수 있다. 그러나 요구 사항에 나와 있는 항목은 모두 시험 항목에 포함시켜야 한다.

<표 2-13> 시험 항목 리스트 예제

순번	시험 항목	test case ID	시험 방법
1	프로그램 실행 시 UI가 설계 화면과 같이 도시되어야 한다.	TC-2-01	검사, 데모
2	이동 도시부에 로봇 아이콘이 도시되어야 하며 현재 상태를 반영하여야 한다.	TC-2-02	데모
3	이동 도시부에 현재 로봇의 X, Y축 좌표가 도시되어야 한다.	TC-2-03	데모
4	로봇의 X축의 이동 속도를 0~10 사이 값으로 설정할 수 있어야 한다.	TC-2-04	데모
5	로봇의 X축 이동 방향을 + 혹은 -로 설정할 수 있어야 한다.	TC-2-05	데모
6	로봇의 Y축의 이동 속도를 0~10 사이 값으로 설정할 수 있어야 한다.	TC-2-06	데모
7	로봇의 X축 이동 방향을 + 혹은 -로 설정할 수 있어야 한다.	TC-2-07	데모
8	로봇의 X축의 위치가 -80이상이거나 80이하일 경우는 초록색 LED가, -80보다 작고 -100이상이거나 80보다 크고 100이하일 경우는 노란색 LED가, -100보다 작거나 100보다 클 경우는 빨강색 LED가 켜져야 한다.	TC-2-08	데모
9	로봇의 Y축의 위치가 -80이상이거나 80이하일 경우는 초록색 LED가, -80보다 작고 -100이상이거나 80보다 크고 100이하일 경우는 노란색 LED가, -100보다 작거나 100보다 클 경우는 빨강색 LED가 켜져야 한다.	TC-2-09	데모
:	:	:	:
00	프로그램 종료 버튼을 누르면 프로그램이 정상적으로 종료되어야 한다.	TC-2-XX	데모

모든 시험 항목을 식별하고 위의 표처럼 작성한다. 단, 위의 시험 항목은 하나의 사례일 뿐이다. 필요한 시험 항목을 분류하고 식별하는 것은 시험자의 취향이나 시험 방법에 따라 다르게 작성될 수 있다.

(2) 테스트 케이스를 작성한다.

위에서 식별한 시험 항목에 대해 각각 테스트 케이스를 작성한다. 테스트 방법은 블랙박스 방법으로 수행하며 테스트 케이스 작성 시 테스트 항목의 성격에 따라 선택스 시험, 등가 분할, 경계값 분석 기법에 따라 절차를 작성한다.

또 상향식 기법으로 시험을 수행할 것인지 하향식 기법으로 수행할 것인지를 결정하고 거기에 맞게 테스트 케이스를 작성한다.

시험 항목 1에 대한 테스트 케이스는 아래와 같이 작성할 수 있다.

<표 2-14> 테스트 케이스 예제1

프로그램명: RoboCon			단계: 통합 시험		
테스트 케이스					
Test Case ID: TC-2-01			Req ID: N/A		
Test Case 설명: 프로그램 실행 시 UI가 설계 화면과 같이 도시되어야 한다.					
테스트 조건: 프로그램이 설치되어 있는 컴퓨터에 로그인 되어 있어야 한다.					
단계	입력값	예상 출력값	실행 결과	Pass/Fail	조치 사항
1	실행	UI 도시			

시험 항목 8에 대한 테스트 케이스는 아래와 같이 여러 경우로 작성할 수 있다. 시험자는 이들 기법 중 하나를 선택할 수도 있고 테스트 케이스를 섞거나 합칠 수도 있다. 테스트 케이스를 어떻게 식별하고 적용하느냐는 전적으로 시험자의 선택이다.

<표 2-15> 테스트 케이스 (등가 분할) 예제2

프로그램명: RoboCon			단계: 통합 시험		
테스트 케이스					
Test Case ID: TC-2-XX			Req ID: R-04		
Test Case 설명: 로봇의 X 축의 위치가 -80이상이거나 80이하일 경우는 초록색 LED가, -80보다 작고 -100이상이거나 80보다 크고 100이하일 경우는 노란색 LED가, -100보다 작거나 100보다 클 경우는 빨강색 LED가 켜져야 한다.					
테스트 조건: 상위 모듈 000 모듈의 테스트 드라이버를 작성해야 한다. 혹은 하위 모듈 000 모듈에 대한 스텝을 작성해야 한다. (시험자 선택)					
단계	입력값	예상 출력값	실행 결과	Pass/Fail	조치 사항

1	25	초록색			
2	-89	노란색			
3	109	빨강색			

<표 2-16> 테스트 케이스 (경계값 분석) 예제2

프로그램 명 : RoboCon			단계 : 통합 시험		
테스트 케이스					
Test Case ID : TC-2-XX			Req ID : R-04		
Test Case 설명 : 로봇의 X 축의 위치가 -80이상이거나 80이하일 경우는 초록색 LED가, -80보다 작고 -100이상이거나 80보다 크고 100이하일 경우는 노란색 LED가, -100보다 작거나 100보다 클 경우는 빨강색 LED가 켜져야 한다.					
테스트 조건 : 상위 모듈 000 모듈의 테스트 드라이버를 작성해야 한다. 혹은 하위 모듈 000 모듈에 대한 스텝을 작성해야 한다. (시험자 선택)					
단계	입력값	예상 출력값	실행 결과	Pass/Fail	조치 사항
1	-101	빨강색			
2	-100	노란색			
3	-99	노란색			
4	-81	노란색			
5	-80	초록색			
6	-79	초록색			
7	79	초록색			
8	80	초록색			
9	81	노란색			
10	99	노란색			
11	100	노란색			
12	101	빨강색			

(3) 요구 사항 추적성 매트릭스를 작성한다.

시험 항목을 식별하고 테스트 케이스를 작성한 후 요구 사항 추적성 매트릭스를 작성하여 작성한 소프트웨어가 모든 요구 사항을 충족하는지 확인한다. 이는 테스트링 과정에서 확인(verification) 과정의 일부이기도 하다.

소프트웨어 요구 사항을 검증하기 위한 모든 테스트 케이스를 매칭한다.

<표 2-17> 요구 사항 추적성 매트릭스 예

소프트웨어 요구 사항		test case	
ID	내용	ID	시험명
R-01	사용자는 X, Y축의 현재 상태값을 알 수 있어야 하며 각 축은 -110~110 사이의 값을 가진다.	TC-2-01	화면 도시
		TC-2-02	로봇의 현재 상태 도시부에 도시
		TC-2-03	로봇의 X, Y축 좌표 도시
		:	:
R-02	사용자는 X축으로의 이동에 대해 '+' 혹은 '-' 방향으로 제어가 가능해야 한다.	TC-2-00	:
R-03	사용자는 Y축으로의 이동에 대해 '+' 혹은 '-' 방향으로 제어가 가능해야 한다.	TC-2-00	:
		:	:
R-04	사용자는 로봇의 상태를 알 수 있어야 한다. 로봇의 각 축의 위치가 -80 이상이거나 80이하일 경우는 초록색 LED가, -80보다 작고 -100이상이거나 80보다 크고 100이하일 경우는 노란색 LED가, -100보다 작거나 100보다 클 경우는 빨강색 LED가 켜져야 한다.	TC-2-08	X축 상태에 따른 LED 표시
		TC-2-09	Y축 상태에 따른 LED 표시
		:	:
		:	:
R-05	사용자는 이동 속도를 1~10/sec까지 1/sec 단위로 설정할 수 있어야 한다.	TC-2-00	:
R-06	사용자는 모든 상태를 초기화할 수 있어야 하며, 초기화 시 X, Y축의 상태는 0이어야 한다.	TC-2-00	:
		:	:
R-07	프로그램 실행 시 CPU 점유율은 최대 20% 사용 메모리는 5MB이하여야 한다.	TC-2-00	:
		:	:

수행 tip

- 프로그램의 규모가 크지 않는 경우 통합 시험은 빅뱅기법을 사용하는 것이 좋다.
- 그러나 학습을 위해서는 상향식 기법 혹은 하향식 기법을 사용해서 통합해가는 과정을 배우는 것이 좋다.

수행 내용 2 / 통합 시험 수행 및 결과보고서 작성하기

재료 · 자료

- 노트 및 필기도구
- 소프트웨어 요구 사항 명세서
- 기타 소프트웨어 설계 산출물 (소프트웨어 설계 명세서, 클래스 다이어그램 등)
- 시험 대상 프로그램(RoboCon) 소스 코드
- 정적 분석 시험 절차서

기기(장비 · 공구)

- 컴퓨터, 윈도우즈 XP 이상
- 사무용 프로그램 (아래한글 or MS Office)
- 프린터
- 인터넷

안전 · 유의 사항

- 공개 소프트웨어를 다운로드하여 사용할 경우 라이선스를 반드시 확인하여 사용 여부를 결정하여야 한다. 일부 소프트웨어는 공개했다가 셰어웨어나 유료 판매 정책으로 수정할 수도 있다.
- 학습을 위해 다운로드한 프로그램은 학습이 완료된 후 학습장의 지침을 확인하여 필요 시 프로그램을 제거해야 한다.

수행 순서

① 시험 절차서를 보고 시험을 준비한다.

앞의 시험 절차서 작성하기의 결과물인 시험 절차서를 확인하고 통합 시험에 필요한 문서 및 장비 등을 준비한다.

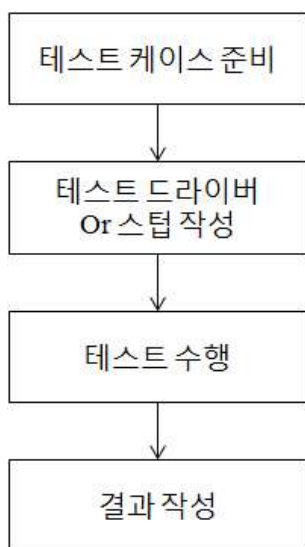
- 노트 및 필기도구
- RoboCon 프로그램

② 통합 시험을 수행한다.

통합 시험의 준비가 완료되었으면, 테스트 케이스에 맞게 시험을 실시한다. 시험의 절차는 테스트 케이스에 준하여 실시하며 1개의 절차가 완료되면 그 결과를 테스트 케이스에 기록한다.

필요 시 테스트 드라이버나 스텝을 작성해서 시험을 해야 한다. 테스트 수행 후 합격/불합격에 대한 결과를 테스트 케이스에 기록하고 시험 결과가 불합격일 경우 자세한 내용 및 프로그래머에게 권고할 사항을 같이 기록한다.

수행 절차는 아래와 같다.



[그림 2-4] 통합 시험 절차

③ 시험 결과보고서를 작성한다.

1. ‘개요’ 를 작성한다.

시험 대상 프로그램의 설명, 시험 목적, 참고 문서 등을 작성한다.

2. 시험 결과를 작성한다.

(1) 시험 결과 종합표를 작성한다.

시험을 한 결과를 종합하여 아래와 같은 표로 작성한다.

<표 2-18> 시험 결과보고서 종합 사례

시험식별자	시험명	합격	불합격	비고
TC-2-01	화면 도시	V		
TC-2-02	로봇의 현재 상태 도시부에 도시	V		
TC-2-03	로봇의 X, Y축 좌표 도시	V		
:	:			
TC-2-08	X축 상태에 따른 LED 표시	V		
TC-2-09	X축 상태에 따른 LED 표시	V		
:	:			
합 계		00	0	0

(2) 세부 시험 결과를 작성한다.

각 테스트 케이스별 세부 시험 결과를 작성한다.

(3) 이슈 사항을 작성한다.

시험 결과가 불합격으로 나왔거나 합격이라도 프로그래머에게 알려야 할 사항이 있다면 이슈 사항으로 정리하여야 한다. 이슈 사항은 되도록 자세하게 작성하여야 한다.

(4) 결론 및 권고 사항을 작성한다.

전체 시험 결과에 대한 결론과 권고 사항을 작성한다.

수행 tip

- 신택스 시험, 등가 분할, 경계값 분석 등의 시험 방법 선택은 시험 항목별로 달라질 수 있으니 적절한 방법을 찾는다.

교수 방법

- 다양한 테스트 케이스를 제시하여 학습자들이 다양한 테스트 케이스를 접할 수 있게 한다.
- 통합 시험에서 실시할 시험 수행 기법에 대해 예시와 함께 자세히 설명한다. 빅뱅 기법, 상향식 기법, 하향식 기법의 적용 사례와 그 외의 기법에 대해서도 설명한다.
- 블랙박스 시험의 다양한 시험 방법에 대해 예시와 함께 장단점을 사례와 함께 설명한다.
- 시험에 오류가 없을 경우, 프로그램 코드에 오류를 거꾸로 삽입하도록 유도하여 오류에 대한 개념을 익히게 한다.
- 테스트 케이스, 시험 절차서, 시험 결과보고서의 양식을 여러 개 준비해서 선택해서 사용할 수 있도록 한다.

학습 방법

- 사전에 테스트 케이스를 작성하는 다양한 방법에 대해 익힌다.
- 통합 시험의 시험 수행 기법의 종류와 방법에 대해 숙지하고 각 기법을 실제로 어떻게 적용할 지에 대해 생각해본다.
- 블랙박스 시험의 다양한 방법에 대해 어떤 경우에 적용할 지와 실제 프로그램에 어떻게 적용할 지를 생각해본다.
- 작성한 예제 프로그램의 시험 시 오류가 없을 경우, 프로그램 코드에 의도적으로 오류를 삽입하여 테스트 케이스의 결과가 어떻게 나오는 지 확인한다.
- 테스트 케이스, 시험 절차서, 시험 결과보고서는 정해진 양식이 있는 것이 아니기 때문에 목적에 맞게 내용을 충실하게 작성하도록 한다.

학습2 평 가

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	평가 항목	성취수준		
		상	중	하
통합기능 시험평가	- 통합기능의 정상적 수행 여부를 판단할 수 있는 시험절차서를 작성할 수 있다.			
	- 통합시험 절차에 따라 단위 소프트웨어와 소프트웨어 컴포넌트를 통합한 프로그램의 운용로직 구현여부를 평가할 수 있다.			
	- 통합기능 시험평가 결과를 보고서로 작성할 수 있다.			

평가 방법

- 포트폴리오

학습 내용	평가 항목	성취수준		
		상	중	하
통합기능 시험평가	- 시험 절차서			
	- 테스트 케이스			
	- 결함 리포트			
	- 시험 결과보고서			

- 문제 해결 시나리오

학습 내용	평가 항목	성취수준		
		상	중	하
통합기능 시험평가	- 요구 사항을 확인을 위한 시험 수행 기법과 시험 방법을 적절하게 선택			
	- 모든 요구 사항에 대해 확인할 수 있도록 테스트 케이스 작성			
	- 프로그래머에게 피드백할 결함 리포트 작성			

• 서술형 시험

학습 내용	평가 항목	성취수준		
		상	중	하
통합기능 시험평가	- 빅뱅 기법, 상향식 기법, 하향식 기법의 구분			
	- 선택스 시험, 등가 분할, 경계값 분석의 선택			
	- 시험 방법에 따른 테스트 케이스의 시험 횟수 선정			

• 평가자 질문

학습 내용	평가 항목	성취수준		
		상	중	하
통합기능 시험평가	- 빅뱅 기법, 상향식 기법, 하향식 기법의 구분			
	- 선택스 시험, 등가 분할, 경계값 분석의 선택			
	- 시험 방법에 따른 테스트 케이스의 시험 횟수 선정			

• 일지/저널

학습 내용	평가 항목	성취수준		
		상	중	하
통합기능 시험평가	- 테스트 케이스 작성 검토 후 수정 반영 여부			
	- 결함 발견 시 처리 과정			
	- 절차서 작성, 시험평가 과정의 기록			

• 작업장 평가

학습 내용	평가 항목	성취수준		
		상	중	하
통합기능 시험평가	- 시험 시 과정 및 결과 특이사항 등 기록			
	- 시험평가 과정 및 결함 발견 시 기록 여부			

피드백

1. 포트폴리오
 - 작성된 테스트 케이스와 시험 결과보고서를 검토하여 미비한 점을 보완하도록 한다.
2. 문제 해결 시나리오
 - 선택한 시험 수행 방법이 왜 적절한 지 혹은 다른 방법이 나은지에 대해 설명하고 수정해서 시험하게 한다.
3. 서술형 시험
 - 선택한 시험 수행 방법이 왜 적절한 지 혹은 다른 방법이 나은지에 대해 설명하고 수정해서 시험하게 한다.
4. 평가자 질문
 - 시험 방법에 따른 테스트 케이스의 시험 횟수 선정하는 룰에 대해 설명한다.
5. 일지/저널
 - 시험 결과보고서 및 결함이 생겼을 때 조치한 결과물 등을 제출하도록 하고 검토 후 필요 없는 항목이 기록되었거나 중요한 항목이 누락되었을 경우 첨삭하여 회신해준다.
6. 작업장 평가
 - 시험 과정에서 이슈 사항 발생 시 혹은 결함 발견 시 기록하는 습관에 대해 강조한다.

학습 1	단위기능 시험평가하기
학습 2	통합기능 시험평가하기
학습 3	신뢰성 시험평가하기

3-1. 신뢰성 시험평가

학습 목표

- 로봇 소프트웨어 신뢰성 시험평가를 위한 시험 절차서를 작성할 수 있다.
- 시험 절차서에 따라 신뢰성 시험평가 항목에 대한 세부 계획 및 적합한 자동화 시험 도구를 채택할 수 있다.
- 신뢰성 시험평가 항목에 대한 세부 계획 및 적합한 자동화 시험 도구를 채택할 수 있다.
- 자동화 시험 도구를 이용하여 시험평가 항목에 대한 신뢰성 시험을 실시할 수 있다.
- 결과보고서는 산출물 문서에 포함하여 추적성을 유지하도록 작성할 수 있다.

필요 지식 /

① 소프트웨어 신뢰성 시험

소프트웨어 신뢰성이란 주어진 환경에서 주어진 시간 동안 오류 없이 동작할 확률을 말한다. 하드웨어와 달리 소프트웨어에 대한 신뢰성 시험은 정적 분석과 동적 분석을 통해 이루어진다.

1. 정적 분석

정적 분석은 소프트웨어를 실행하지 않은 상태에서 결함을 검출하는 방식으로 사람이 직접 오류를 찾아내는 수동 방식과 자동화 도구를 이용하여 오류 패턴을 찾아내는 자동 방식이 있다.

코드리뷰가 사람에 의해 이루어지는 수동 방식이라면 자동화 도구를 이용하면 코딩 규칙 검증과 실행 시간 오류(runtime error) 검출, 코드의 복잡도 계산, 실행되지 않는 데드코드 등을 검출할 수 있다.

실행 시간 오류(runtime error) 검출 시험은 프로그램의 수행 경로를 추적하면서 프로그램 실행에 영향을 미치는 실행 정지, 오동작, 메모리 누수 등의 런타임 오류를 찾아내는 시험이다. 대표적인 실행 시간 오류의 목록인 CWE-658/659의 목록은 아래 표와 같다.

<표 3-1> CWE-658/659 실행 시간 오류 목록

순번	Defect type	순번	Defect type
1	Buffer Overrun	23	Overlapping Memory Regions
2	Buffer Underrun	24	Redundant Condition
3	Cast Alters Value	25	Return Pointer To Freed
4	Dangerous Function Cast	26	Return Pointer To Local
5	Division By Zero	27	Type Mismatch
6	Double Close	28	Type Overrun
7	Double Free	29	Type Underrun
8	Empty If Statement	30	Uninitialized Variable
9	File System Race Condition	31	Unreachable Call
10	Format String	32	Unreachable Computation
11	Free Non-Heap Variable	33	Unreachable Condition
12	Free Null Pointer	34	Unreachable Control Flow
13	Integer Overflow of Allocation Size	35	Unreachable Data Flow
14	Ignored Return Value	36	Unreasonable Size Argument
15	Leak	37	Unused Value
16	Misaligned Object	38	Use After Colse
17	Missing Return Statement	39	Use After Free
18	Missing Return Value	40	Useless Assignment
19	Negative File Descriptor	41	Varargs Function Cast
20	No Space For Null Terminator	42	Deadlock
21	Null Pointer Dereference	43	Double Lock
22	Null Test After Dereference	44	Double Unlock

코딩 규칙에 대한 설명은 ‘[2] 코딩 규칙’ 에서 따로 설명한다.

2. 동적 분석

동적 분석은 소프트웨어를 직접 실행시키면서 검증하는 방법으로 동적 분석의 시험평가 기준은 코드 커버리지(code coverage)이다.

코드 커버리지란 프로그램 코드가 시험을 수행했는가를 백분율로 나타내는 값으로 프로그램의 성격과 종류에 따라서 목표값을 다르게 적용하는 것이 일반적이다. 그리고 동적 분석은 그 특성상 시험을 하는 과정에 비용과 시간이 많이 들어가므로 소프트웨어의 특성과 시험 비용 및 기간을 고려하여 코드 커버리지의 목표값을 적절히 설정해야 한다.

② 코딩 규칙

코딩 규칙이란 코드의 가독성이나 신뢰성을 향상시키기 위해 정한 규칙으로 규칙을 정한 산업 표준 혹은 기관에 따라 코딩 규칙이 조금씩 다르다. 컴파일러가 찾아낼 수 있는 문법적인 오류와 다르다는 면에서는 코딩 스타일과 비슷하나 그와는 달리 정해진 규칙대로 코딩을 수행했는지 걸러내는 틀이 존재하는 것이 코딩 스타일과 다른 점으로 신뢰성을 위해 정해놓은 강제 가이드라인으로 이해하면 된다.

행정안전부에서는 코딩 규칙에 대해 다음과 같이 정의하고 있다. “코드 작성에 관한 지침, 규칙 및 규정의 집합을 말한다. 변수 명명, 코드 들여쓰기, 괄호나 키워드의 위치 정하기 등 주의를 해서 작성하여 하는 문장과 함수, 각 언어의 특성 및 개발환경으로 인해 발생하는 지침을 포함한다. 코딩 규칙은 일반적으로 코드의 가독성, 확정성, 신뢰성을 향상시키기 위한 목적으로 사용한다.”

예를 들어 아래와 같이 C 언어로 작성된 코드를 살펴보자.

```
int v; v++;  
printf( "Variable = %d\n" , v);
```

이 코드는 실행 측면에서 보면 문법에 문제가 없어 컴파일도 되고 프로그램 실행에도 문제가 없다. 그러나 몇몇 주요한 코딩 규칙의 관점으로 보면 ‘1줄에 2개의 statement를 썼다는 점’과 ‘변수를 선언하고 초기화 하지 않은 점’의 2가지 규칙을 위배했다고 지적할 것이다.

코딩 규칙은 산업별, 회사별, 기관별로 조금씩 다르게 정해져 있다. 그 중 중요한 코딩 규칙 몇 가지를 살펴보자.

1. MISRA-C, C++

MISRA-C는 자동차산업소프트웨어신뢰성협회(MISRA, motor industry software reliability association)에서 개발된 C 프로그래밍에 대한 개발표준으로 초판은 1998년도에 출간된 MISRA-C:1998 이며, MISRA-C:2004를 거쳐 현재는 MISRA-C:2012까지 출간되었다.

MISRA-C:2012는 총 143개의 룰이 있으며, 각 룰은 ‘필수’, ‘요구’, ‘권고’로 구분된다. MISRA는 반드시 지켜야 하거나, 지키면 좋은 코딩 규칙으로 이루어져 있는데, MISRA C:2004의 경우는 전체 141개의 코딩 규칙 중에서 반드시 지켜야 할 규칙이 121개, 권고되는 규칙은 20개로 이루어져 있다.

이 표준 문서는 <http://www.misra.or.uk> 에서 구매할 수 있다.

2. SEI CERT coding standard

전미 소프트웨어협회(software engineering institute)의 CERT(computer emergency response team)에서 만든 CERT C 코딩 표준은 수많은 취약성을 분석하여 공통의 프로그래밍 오류를 찾아내고 안전하지 않은 코딩에 대응되는 안전한 코딩 방법을 구축하였다. 이 표준은 89개 규칙과 132개의 권고로 구성되어 있다.

CERT C 코딩 표준에서는 각 분류별로 00-29번은 권고를 30번 이후는 규칙을 나타낸다.

3. CWE-658/659

CWE(common weakness enumeration)는 소프트웨어 보안 및 품질 강화를 위해 개발 시에 참고할 수 있도록 전 세계 SW 취약점을 표준화한 목록으로 미국 국토안전부와 미국 국립표준기술연구소(NIST) 산하 비영리 기관인 MITRE에서 후원하고 있다.

CWE는 CVE 리스트의 다양한 실제 취약성을 기반으로 CWE 커뮤니티의 많은 소스와 예제를 통해 취약성 리스트를 만들어 나간다. CWE-658/659는 MITRE에서 C, C++언어로 소프트웨어 개발 시 공통적으로 취약한 결함을 식별한 목록을 나타낸다.

4. OWASP

OWASP(the open web application security project)는 오픈소스 웹 애플리케이션 보안 프로젝트이다. 주로 웹에 관한 정보 노출, 악성 파일 및 스크립트, 보안 취약점 등을 연구하며, 10대 웹 애플리케이션의 취약점 (OWASP TOP 10)을 발표했다.

5. DAPA SCR-G 2011 C, C++

한국의 방위사업청에서 만든 코딩 표준으로 신뢰성이 무엇보다 중요한 무기체계의 소프트웨어 개발 시 적용하는 표준으로 개발하였다. 주요한 규칙 몇 가지를 살펴보면 아래와 같다.

(1) 주석 및 명명 규칙

<표 3-2> DAPA의 주석 및 명명 규칙

구분	내 용
주석 규칙	1) C 프로그램 경우 “/* ~ */”, C++ 프로그램 경우 “/* ~ */” 또는 “// ~”를 사용한다(단, C 프로그램의 경우 컴파일러가 “// ~”를 지원할 경우에 사용할 수 있다). 2) 주석 내에는 작성자, 날짜, 수정 이력, 기능 설명, 설계 문서와 연계되는 정보 등의 필수 항목은 반드시 포함한다. 3) 주석 내에는 Nested Comment 삽입해서는 안 된다. 4) 기타 주석 작성은 개발자 특성에 맞는 규칙을 사용하되 조직 차원에서

	의 통일된 지침을 수립하여 적용한다.
	5) 소스 코드 내 해당 내용을 자세하고 정확하게 설명할 수 있는 주석을 작성하고, 작성 된 주석은 전체 소스 코드의 30% 이상으로 한다.
파일명	가. 함수/클래스일 경우, 파일명을 일치시키고 복수일 경우 파일명은 기능을 잘 설명해 줄 수 있는 명칭으로 한다. 나. 파일의 첫 글자는 대문자 또는 소문자로 통일하여 표기한다. 다. 파일을 잘 설명할 수 있는 이름으로 파일명을 작성한다.
함수/ 클래스/ 메소드명	가. 기능을 잘 표현할 수 있는 명칭을 사용한다. 나. 함수 생성 시 “동사 + (명사)” 순으로 한다. 다. 클래스 생성 시 “ClassName” 형태로 조합하되 첫 단어의 글자는 대문자로 작성한다. 라. 클래스, 함수, 메소드명을 작성할 경우, 실제 기능을 유추할 수 있는 단어를 조합하여 작성한다.
변수명	가. Scope, Linkage를 구분할 수 있어야 하며, 각각 Internal, Extern을 구분하여 작성한다. 예) “Pointer 변수는 p, Global 변수는 g, Static 변수는 s, Count 변수는 n” 으로 표기 나. 변수명의 첫 글자는 반드시 문자를 사용하고 31자를 넘기지 않는다. 다. 변수명은 헝가리언, 카멜, 파스칼 표기법 또는 조직 내부 작성 지침에 따라 사용한다.
매크로명	가. 매크로명은 전체를 대문자 또는 “_” 를 사용한다. 나. 파일, 매크로명은 표준에 있는 예약어를 사용하지 않는다. 예) _변수명, scanf, printf 등

(2) 프로그램 작성 규칙

<표 3-3> DAPA의 프로그램 작성 규칙

구분	주요 내용	개수
C, C++ 공통 적용 (45개)	스타일	한 라인씩 구성, goto문, 다중 return문 자제 등 11
	초기화	변수, 문자열 배열, 포인터 초기화 등 3
	식별자	전역변수 중복 식별자 자제 등 4
	조건식	float 자료형 비교, 조건문 결과 항상 True, False 등 5
	변 환	함수 변환, 데이터형 변환, 상수형 자료 변환 등 8
	포인터/배열	Null 포인터 참조, 배열 인덱스 초과값 사용 등 5
	연산자	0 나누기 연산, side effect, 음수 shift 자제 등 9
C 전용 적용	scanf, malloc, 함수 파라미터 등	5
C++ 전용 적용	throw, virtual function, exception 처리, 생성자, 소멸자 등	15

③ 자동화 도구

소프트웨어 신뢰성 시험을 사람이 직접 수행하려면 많은 인력과 비용이 들어갈 뿐만 아니라 실수를 배제할 수 없기 때문에 자동화 도구를 이용하는 것이 도움이 많이 된다. 하지만 테스트 도구를 사용하는 것은 장점뿐 아니라 단점 역시 포함하고 있으므로 테스트 도구에 대한 이해를 통한 적절한 도구를 선택하는 것이 필요하다.

1. 자동화 도구의 장단점

(1) 장점

- 테스트 데이터의 재입력 같은 반복 작업의 자동화
- 테스트 드라이버 및 스텝 같은 테스트 코드 자동 생성
- 코딩 규칙이나 복잡도 계산 등 정의된 평가에 대한 신속하고 정확한 수행
- 성능에 대한 통계와 그래프 등 테스트 정보의 자동 생성 기능

(2) 단점

- 상용 소프트웨어의 경우 투자 비용 및 유지 관리 비용이 높음
- 복잡한 도구의 사용에 대한 학습 필요
- 도입 후 프로세스 적용에 대한 시간, 비용 필요

2. 공개 소프트웨어

요즘은 고가의 상용 소프트웨어뿐만 아니라 테스트 자동화를 위한 공개용 테스트 자동화 도구들이 많이 소개되어 있으니 간단히 살펴보자.

(1) 정적 분석 도구

소스 코드에 대한 코딩 표준/스타일, 복잡도 및 잔존 결함을 발견하기 위하여 사용하는 도구

<표 3-4> 공개용 정적 분석 도구

분류	제품명	세부 정보
결함 예방/ 발견	pmd(cpd)	개요 자바 소스 코드에 대한 잠재적인 문제에 대한 분석
		지원 환경 Linux, Windows
		개발도구 지원 Eclipse, NetBeans 등
		홈페이지 http://pmd.sourceforge.net/
	findbugs	개요 자바 소스 코드에 대한 잠재적인 문제에 대한 분석
		지원 환경 Cross-Platform
		개발도구 지원 Eclipse, NetBeans 등
		홈페이지 http://findbugs.sourceforge.net/
	cppcheck	개요 C/C++ 소스 코드에 대한 잠재적인 문제에 대한 분석
		지원 환경 Windows
		개발도구 지원 Eclipse, gedit 등

코딩 표준	sonar	홈페이지	http://cppcheck.sourceforge.net/	
		개요	C/C++, Java 등 다양한 언어 지원 및 PMD, CheckStyle 등 플러그인을 통하여 확장 가능	
		지원 환경	Cross-Platform	
		개발도구 지원	eclipse	
	checkstyle	홈페이지	http://www.sonarsource.org/	
		개요	자바 프로그램에 대한 코딩 표준 준수 검사 도구로, 다양한 개발도구에 통합하여 활용 가능	
		지원 환경	Cross-Platform	
		개발도구 지원	Ant, Eclipse, NetBeans 등	
	N'SIQ CppStyle	홈페이지	http://checkstyle.sourceforge.net/	
		개요	C/C++ 프로그램 언어에 대한 코딩 표준 준수 검사 도구	
지원 환경		Cross-Platform		
개발도구 지원		Visual Studio, eclipse		
코딩 표준	StyleCop	홈페이지	http://dev.naver.com/projects/nsiqcppstyle/	
		개요	C# 프로그램 언어에 대한 코딩 표준 준수 검사 도구	
		지원 환경	Windows	
		개발도구 지원	Visual Studio	
	cpplint	홈페이지	http://stylecop.codeplex.com/	
		개요	구글에서 사용하고 있는 C++ 스타일 가이드 준수 검사 도구로 CLI(파이썬) 형태로 지원됨	
		지원 환경	Cross-Platform	
		개발도구 지원	-	
	코드 복잡도	ccm	홈페이지	http://google-styleguide.googlecode.com/svn/trunk/cppguide.xml#cpplint
			개요	소스 코드 복잡도 분석 도구이며, Linux, Mac환경에서는 CLI 형태로 지원됨
지원 환경			Cross-Platform	
개발도구 지원			Visual Studio	
eclipsemetrics		홈페이지	http://www.blunck.info/ccm.html	
		개요	소스 코드 복잡도 분석 소스 코드 통계 정보 제공 도구	
		지원 환경	Cross-Platform	
		개발도구 지원	Eclipse	
sourcemonitor		홈페이지	http://www.stateofflow.com/projects/16/eclipsemetrics	
		개요	윈도우 기반 소스 코드 복잡도 분석 도구	
		지원 환경	Windows	
		개발도구 지원	-	
cobertura		홈페이지	http://www.campwoodsw.com/sourcemonitor.html	
		개요	자바언어에 대한 소스 코드 복잡도 분석 및 커버리지 측정	
		지원 환경	Cross-Platform	

javancss	개발도구 지원	Ant, Maven
	홈페이지	http://cobertura.sourceforge.net/
	개요	자바언어에 대한 소스 코드 복잡도 분석 도구, CLI 형태로 지원됨
	지원 환경	Cross-Platform
	개발도구 지원	Ant, Jacob
	홈페이지	http://www.kclee.de/clemens/java/javancss/

출처 : http://www.oss.kr/oss_repository6/69515 2016. 08. 05. 검색

(2) 동적 분석 도구

프로그램을 실행하여, 코드 내에 존재하는 메모리 누수, 쓰레드 결함 등을 분석하기 위하여 사용하는 도구

<표 3-5> 공개용 동적 분석 도구

제품명	세부 정보	
Avalanche	개요	Valgrind 프레임워크 기반으로 구현되었으며, 프로그램에 대한 결함 및 취약점 동적 분석 도구
	지원 환경	Linux, Android
	개발도구 지원	-
	홈페이지	http://code.google.com/p/avalanche/
Valgrind	개요	프로그램 내에 존재하는 메모리 및 쓰레드의 결함을 발견하는 동적 분석 도구
	지원 환경	Cross-Platform
	개발도구 지원	Eclipse, NetBeans 등
	홈페이지	http://valgrind.org

출처: http://www.oss.kr/oss_repository6/69515 2016. 8. 5. 검색

(3) 테스트 자동화 프레임워크

단위 테스트, 통합 테스트 등 테스트 단계별 자동화 도구를 활용한 기능 테스트 도구

<표 3-6> 공개용 동적 분석 도구

제품명	세부 정보	
xUnit	개요	java(Junit), C++(Cppunit), .Net(Nunit) 등 다양한 언어를 지원하는 단위 테스트 프레임워크
	지원 환경	각각의 도구별 지원 환경 상이
	개발도구 지원	eclipse 등

STAF	홈페이지	http://www.junit.org/ http://sourceforge.net/apps/mediawiki/cppunit/index.php?title=Main_Page http://www.nunit.org/
	개요	서비스 호출, 컴포넌트 재사용 등 다양한 환경을 지원하는 테스트 프레임워크
	지원 환경	Cross-Platform
	개발도구 지원	eclipse
FitNesse	홈페이지	http://staf.sourceforge.net/
	개요	웹기반 테스트 케이스 설계/실행/결과 확인 등을 지원하는 테스트 프레임워크
	지원 환경	Cross-Platform
	개발도구 지원	eclipse
NTAF	홈페이지	http://fitnesse.org/
	개요	NHN 테스트 자동화 프레임워크이며, STAF와 FitNesse를 통합
	지원 환경	Cross-Platform
	개발도구 지원	eclipse, Maven 등
Selenium	홈페이지	http://dev.naver.com/projects/ntaf/
	개요	다양한 브라우저 지원 및 개발언어를 지원하는 웹 애플리케이션 테스트 프레임워크
	지원 환경	Cross-Platform
	개발도구 지원	-
watir	홈페이지	http://seleniumhq.org/
	개요	Ruby 기반 웹애플리케이션 테스트 프레임워크
	지원 환경	Cross-Platform
	개발도구 지원	-
	홈페이지	http://watir.com/

출처 : http://www.oss.kr/oss_repository6/69515 2016. 08. 05. 검색

④ 코드 커버리지

100 % 코드 커버리지란 의미는 모든 소스 코드를 한 번 이상 시험을 수행했음을 의미하는 것이다. 그러나 복잡한 프로그램의 경우 100 % 코드 실행률을 달성하기란 매우 어려우며 또 코드 실행을 했다고 해서 에러가 없는 것은 아니다.

코드 커버리지는 여러 가지 종류가 있다.

1. 문장 커버리지(statement coverage)

문장 커버리지는 말 그대로 코드 문장의 수행 여부를 판단한다. 코드의 수행 여부만을 판

단하기 때문에 조건문에서는 항상 True만 존재하는지 False인 경우도 존재하는지 확인되지 않는다. 아래 코드에서는 4번째 Statement(ss=1;)가 실행되지 않는다.

```
int Test(int a, int b) {
    int ss=0;
    if(a==1 && b==1) {
        ss=1;
    }
    return ss;
}
```

테스트 케이스를 작성하기가 매우 쉬우나 보장성이 가장 낮다. 이 경우 테스트 케이스는 한 가지이면 충분하다.

2. 분기 커버리지(branch coverage)

분기 커버리지는 조건문이 있을 경우 그 조건문에 True와 False인 경우가 각각 존재하는지를 검출한다.

그러므로 위 코드의 조건문 'if(a==1 && b==1)' 은 전체 4가지 경우 중 Result가 False인 3가지 경우를 모두 하나의 False로 검출하므로 조건문 내부 각각의 조건에 대한 커버리지는 측정되지 않는다. 그러므로 아래 표의 3, 4번은 실행하지 않는다.

<표 3-7> 분기 커버리지의 테스트 케이스

번호	a==1	b==1	Result
1	True	True	True
2	True	False	False
3	False	True	False
4	False	False	False

테스트 케이스 2가지로 커버리지를 채울 수 있다.

3. 변형다중조건 커버리지(modified condition/decision condition)

MD/DC는 조건문 내에서도 각각의 조건문의 수행 여부를 검출하며, 프로그램의 수행 과정에서 True와 False가 각각 존재하는지 검출한다.

MC/DC 조합 결정 방법은 아래와 같다.

- 모든 결정 포인트 내의 전체 조건식은 적어도 한 번 모든 가능한 결과값(True, False)을 갖도록 한다.

- 결정 포인트 내의 모든 개별 조건식은 적어도 한 번 모든 가능한 결과값(True, False)을 갖도록 한다.
- 결정 포인트에 있는 각각의 개별 조건식은 다른 개별 조건식에 영향을 받지 않고 그 결정 포인트의 결과값에 독립적으로 영향을 준다.

각 조건문 내에서 가능한 결과값을 가지려면 Result와 a, b가 각각 True와 False가 한 번씩 나오면 된다. 따라서 4번 조건은 실행하지 않고 테스트 케이스 3가지가 나온다.

<표 3-8> MC/DC의 테스트 케이스

번호	a==1	b==1	Result
1	True	True	True
2	True	False	False
3	False	True	False
4	False	False	False

수행 내용 1 / 신뢰성 시험 절차서 작성하기

재료 · 자료

- 노트 및 필기도구
- 소프트웨어 요구 사항 명세서
- 기타 소프트웨어 설계 산출물 (소프트웨어 설계 명세서, 클래스 다이어그램 등)
- 시험 대상 프로그램(RoboCon) 소스 코드

기기(장비 · 공구)

- 컴퓨터, 윈도우즈 XP 이상
- 사무용 프로그램 (아래한글 or MS Office)
- 프린터
- 인터넷

안전 · 유의 사항

- 시험 절차서 작성 시 시험의 순서를 잘 나열하여야 한다. 그렇지 않으면 필요 없는 절차를 재수행하는 등 비효율적으로 될 수 있다.

수행 순서

① 시험의 개요와 참고 자료를 작성한다.

‘학습 1 단위기능 시험평가’ 시 작성하였던 RoboCon 프로그램을 이용하여 정적 분석과 동적 분석을 수행하는 시험의 개요를 작성한다.

개요에는 문서명과 프로그램명, 시험의 목적 등을 기록한다.

참고 자료에는 요구 사항과 설계 문서 그리고 단위 시험 결과보고서, 통합 시험 결과보고서 등을 기록한다.

② 시험 준비 사항을 기록한다.

시험 준비 사항에는 신뢰성 시험을 다음 위한 시험 형상, 준비해야 할 소프트웨어, 하드웨어 목록, 시험 장비 등을 기록한다.

준비해야 할 소프트웨어 목록에는 정적 분석용 툴 CPPCheck를 포함시킨다.

③ 신뢰성 시험 절차를 작성한다.

1. 시험의 종류와 방법을 결정한다.

정적 분석으로는 코딩 규칙 검증 및 실행 시간 오류 분석, 동적 분석으로는 코드 커버리지 확인 시험을 추가한다.

2. 코딩 규칙

코딩 규칙 검증은 ‘DAPA SCR-G 2011 C, C++’ 를 적용하도록 하고 자동화 도구 없이 검사자가 코드를 직접 보면서 수행하도록 한다.

3. 실행 시간 오류 분석

실행 시간 오류 분석은 공개 소프트웨어인 자동화 도구를 사용하여 수행하도록 계획한다. 자동화 도구는 CPPCheck를 사용한다.

4. 코드 커버리지 목표값 설정

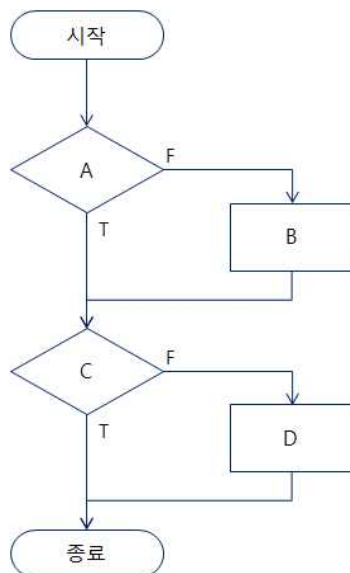
코드 커버리지 확인을 위한 공개 소프트웨어는 대개 통합개발환경(IDE)에 플러그인 형태로 설치되기 때문에 Visual Studio나 Eclipse같은 개발환경이 갖추어지지 않은 컴퓨터에서 수행하기가 어렵다. 검사자가 코드를 직접 확인 및 로깅을 하면서 수행하도록 계획한다.

코드 커버리지의 목표값은 아래와 같이 설정한다.

- 문장 커버리지: 100 %
- 분기 커버리지: 100 %

④ 테스트 케이스를 작성한다.

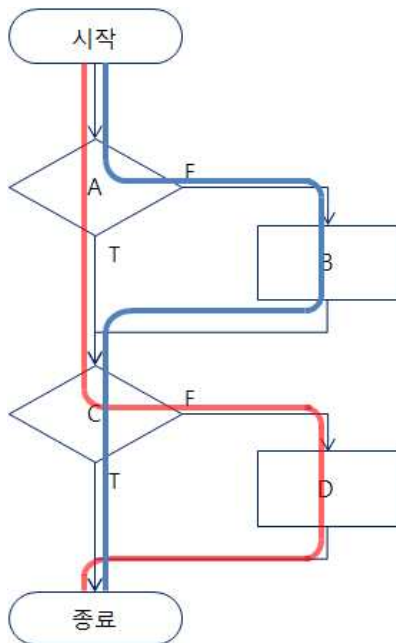
테스트 케이스를 작성하기 전 코드 커버리지의 목표를 세운다. RoboCon 프로그램은 그리 규모가 크지 않으니 분기 커버리지 100 %를 목표로 테스트 케이스를 작성한다.



[그림 3-1] 프로그램 흐름 (예)

만일, 프로그램의 흐름이 위와 같이 되어 있고 분기 커버리지 100 %를 달성하기 위해서는 테스트 케이스를 2개를 작성해야 한다.

- 테스트 케이스 1: 시작 → A(T) → C(F) → D → 종료 (파란색)
- 테스트 케이스 2: 시작 → A(F) → C(T) → D → 종료 (빨간색)



[그림 3-2] 분기 커버리지 테스트 케이스

테스트 케이스를 작성하기 위해서는 시험 대상 프로그램에 대해 클래스 다이어그램과 같은 설계 문서와 프로그램 로직, 코드의 흐름을 정확히 이해해야 한다. 또 조건문과 분기문 등을 확인하여 제어 흐름을 만들고 각 분기점에서 True와 False의 경우를 모두 통과하도록 테스트 케이스를 설계하여야 한다.

마지막으로 모든 테스트 케이스를 검토하여 분기 커버리지 100 %가 될 수 있도록 테스트 케이스가 다 작성되었는지 확인한다.

수행 내용 2 / 정적 분석 수행하기

재료 · 자료

- 노트 및 필기도구
- 소프트웨어 요구 사항 명세서
- 기타 소프트웨어 설계 산출물 (소프트웨어 설계 명세서, 클래스 다이어그램 등)
- 시험 대상 프로그램(RoboCon) 소스 코드
- 정적 분석 시험 절차서

기기(장비 · 공구)

- 컴퓨터, 윈도우즈 XP 이상
- 사무용 프로그램 (아래한글 or MS Office)
- 프린터
- 인터넷

안전 · 유의 사항

- 공개 소프트웨어를 다운로드하여 사용할 경우 라이선스를 반드시 확인하여 사용 여부를 결정하여야 한다. 일부 소프트웨어는 공개했다가 셰어웨어나 유료 판매 정책으로 수정할 수도 있다.
- 학습을 위해 다운로드한 프로그램은 학습이 완료된 후 학습장의 지침을 확인한 후 필요 시 프로그램을 제거해야 한다.

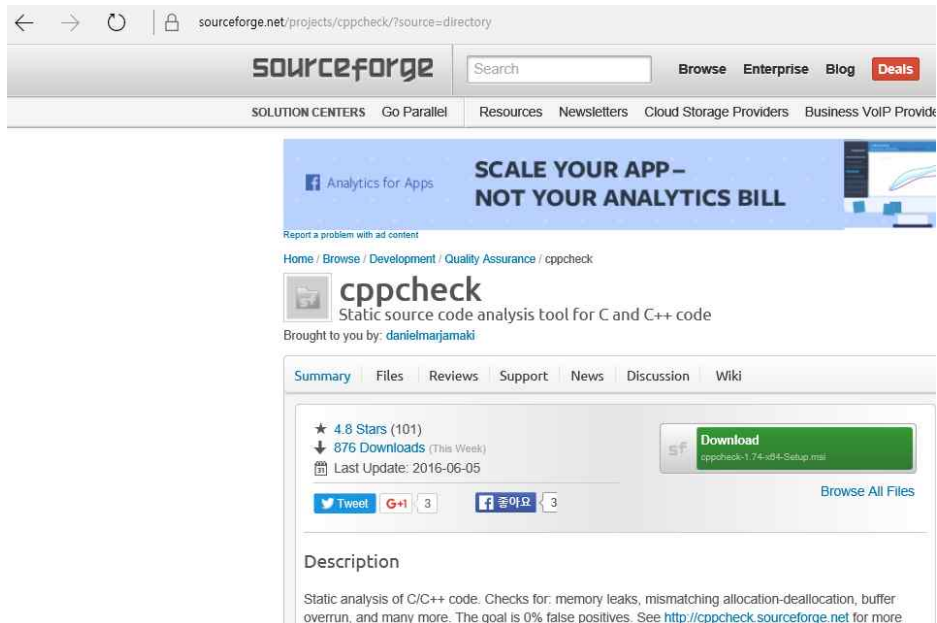
수행 순서

① 정적 분석용 공개 소프트웨어를 설치한다.

정적 분석을 수행하기 위해서는 정적 분석용 툴이 있어야 하며, 대상 소프트웨어가 있어야 한다. 정적 분석용 툴은 공개 소프트웨어 중 CPPCheck를 다운로드하여 사용하며 대상 소프트웨어는 ‘학습 1 단위기능 시험평가’ 시 작성하였던 RoboCon 프로그램을 이용한다.

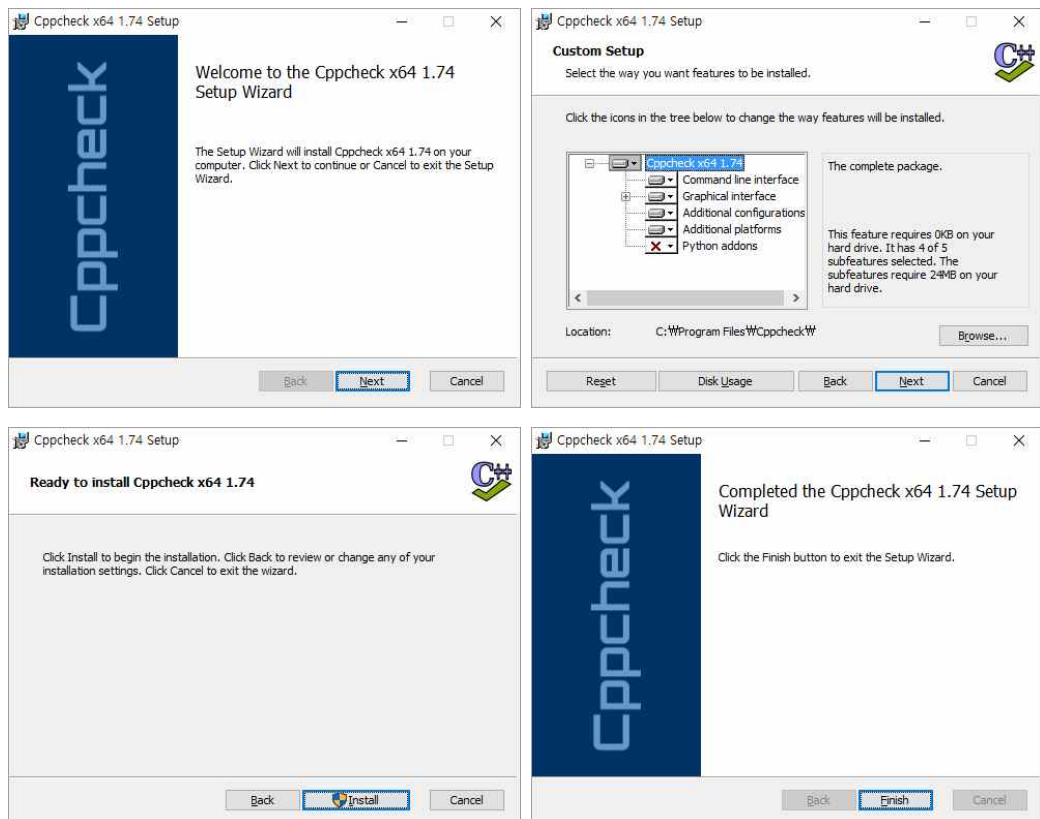
1. 아래 웹사이트로 이동하여 CPPCheck를 다운로드한다.

(<http://cppcheck.sourceforge.net/>)



출처 : <http://cppcheck.sourceforge.net/> 2016. 08. 02. 스크린샷.
[그림 3-3] CPPCheck 다운로드 화면

2. 다운로드한 파일을 설치한다.



[그림 3-4] CPPCheck 설치 과정

3. 프로그램 설치가 안 될 경우 온라인으로 검사한다.

만일, 프로그램을 설치할 수 없다면 아래의 사이트로 이동하여 온라인으로 코드를 검사할 수도 있다. 단, 온라인 데모를 이용할 경우 사용에 불편함이 있다.

(<http://cppcheck.sourceforge.net/demo/>)

② 예제 프로그램 작성하여 정적 분석한다.

CPPCheck 도구의 간단한 사용법과 정적 분석을 수행하는 과정을 확인하기 위해 간단한 샘플 프로그램을 작성하여 실행해본다.

1. 예제 프로그램을 작성한다.

메모장을 열어 아래와 같이 코딩한 후 file1.c로 저장한다.

```
void aaa(char* str)
{
    char* buf =new char[8];
    strcpy(buf, str);

    FILE* file = fopen("sample.txt","w");
    if(!file)
        return;

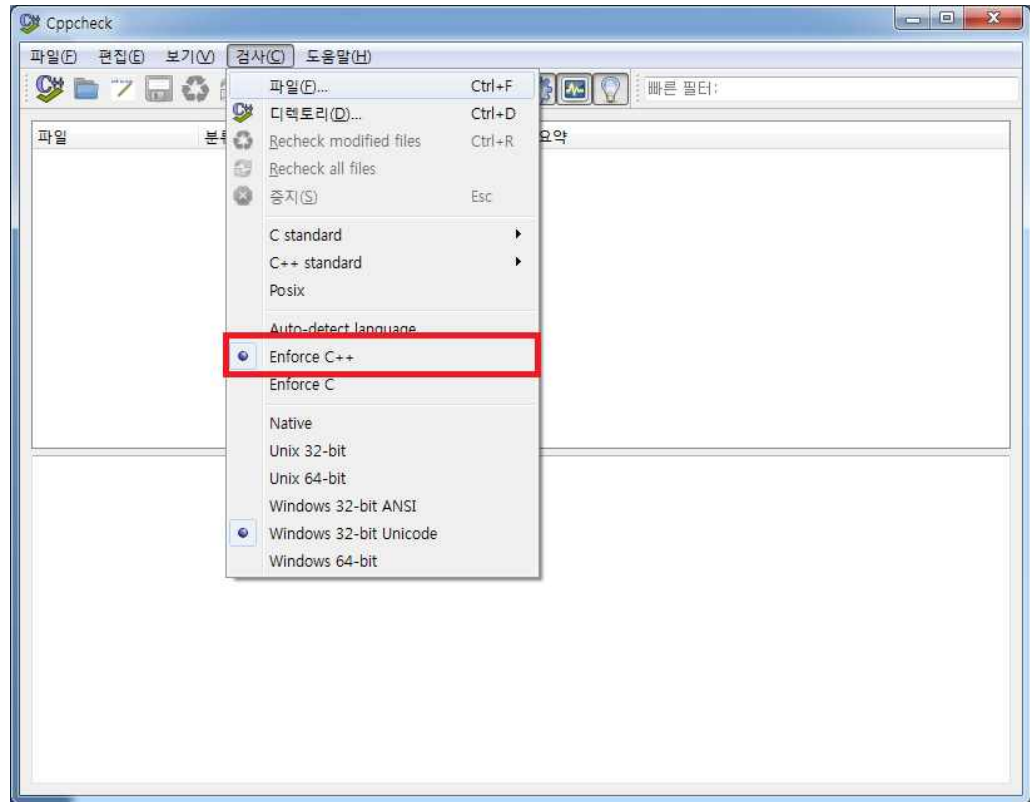
    for(char* c=buf; *c; ++c)
        fputc((int)*c, file);

    delete buf;
}
```

2. CPPCheck 프로그램으로 분석한다.

(1) CPPCheck 프로그램을 열고 아래와 같이 설정한다.

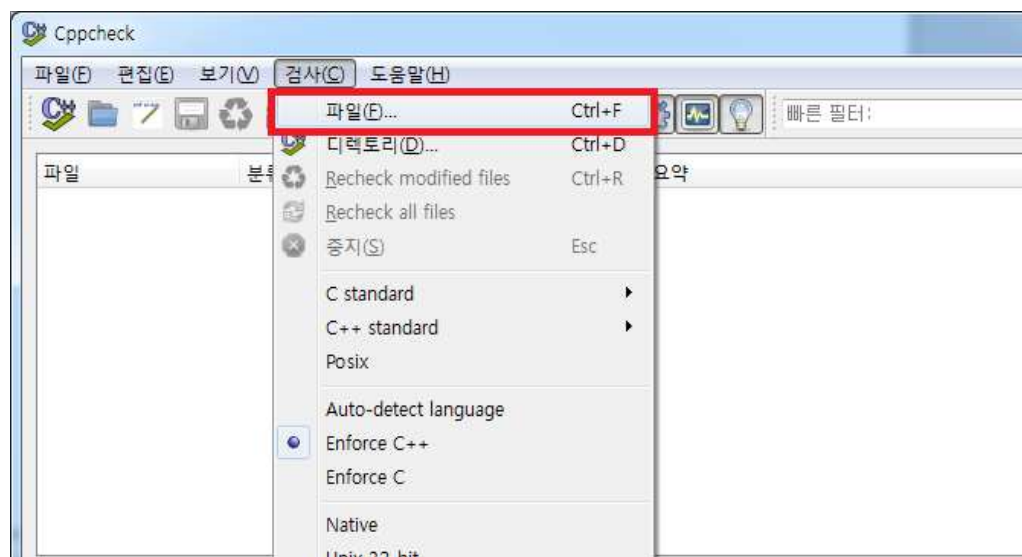
검사 메뉴를 선택 후 'Enforce C++'을 선택한다. 이것은 검사 시 프로그램 언어를 C++ 기준으로 검사한다는 뜻이다. 프로그램 언어를 C로 설정하여 검사를 하면 결과가 달라질 수 있다.



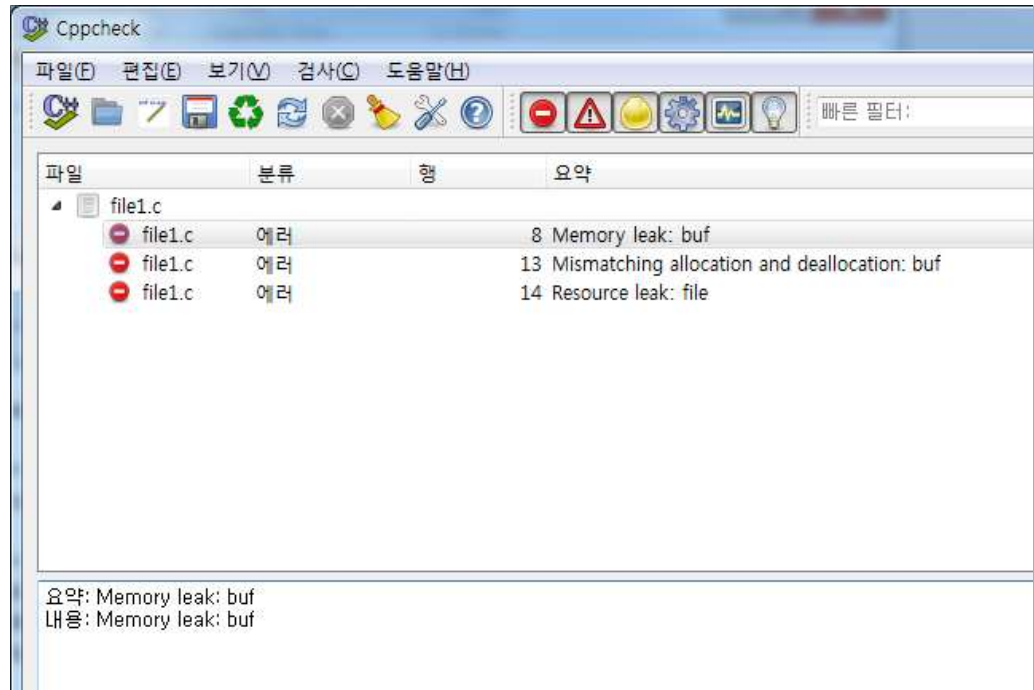
[그림 3-5] Cppcheck 프로그램 언어 설정

(2) 검사를 수행한다.

검사-파일을 선택하여 저장된 file1.c 파일을 연다. 파일을 열면 바로 검사를 수행하고 그 결과를 아래 창에 도시하여 준다.



[그림 3-6] 검사-파일 메뉴



[그림 3-7] 검사 결과 - 오류 발견

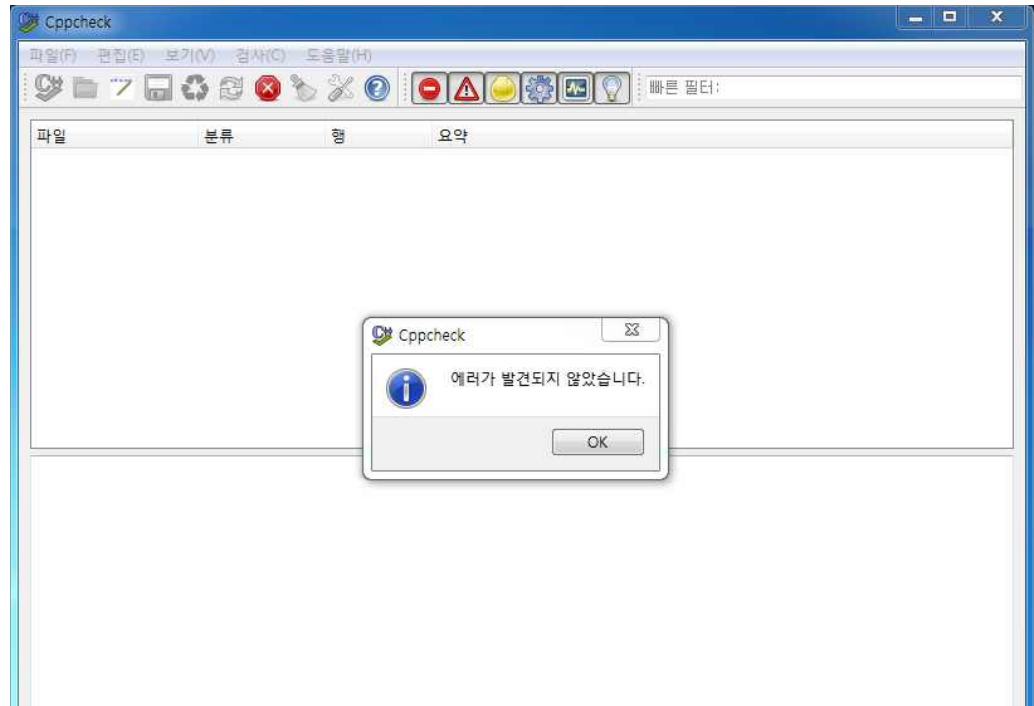
에러가 있는 각 항목을 선택하면 아래 창에 에러의 내용을 보여준다.

Cppcheck 가 검출하는 에러는 아래와 같다.

- 메모리 및 리소스 Leaks
- Out of Bound 체크
- 사용하지 않는 클래스의 Private Function 체크
- 표준 템플릿 라이브러리(STL)에서 허용하지 않는 사용 체크
- 초기화하지 않은 변수 체크
- 기타

(3) 오류를 수정한다.

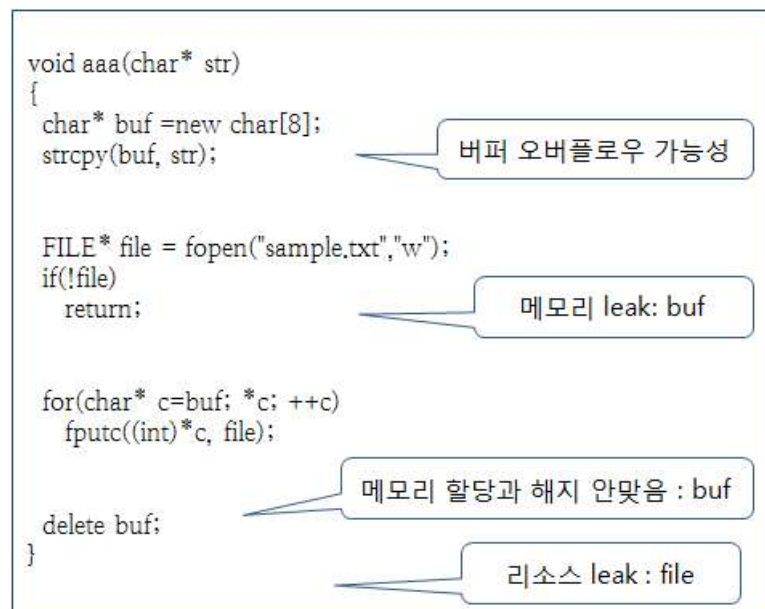
각 오류 사항을 확인하고 수정한다. '검사-Recheck all files' 메뉴를 선택하고 재검사하여 아래와 같이 '에러가 발견되지 않았습니다.' 라는 메시지가 뜨면 모든 오류가 수정된 것이다.



[그림 3-8] 검사 결과 - 에러 없음

(4) 코드리뷰를 통해 자동 검사 툴의 결과와 비교한다.

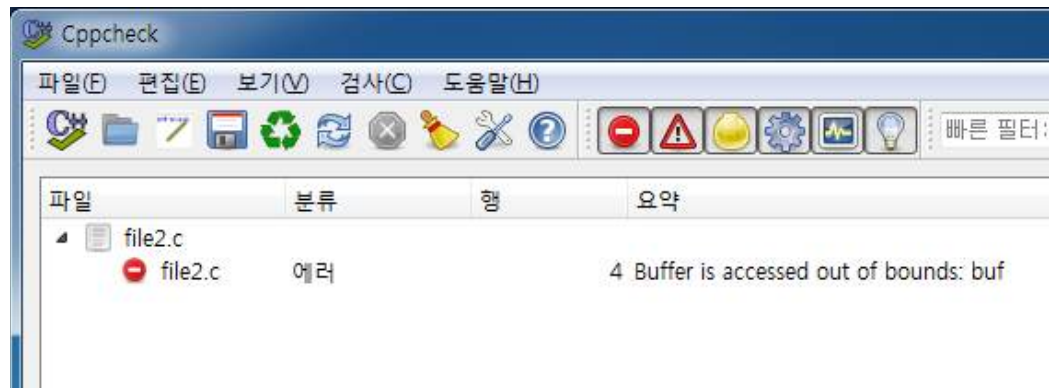
file1.c 코드를 자세히 살펴보면 4곳에서 문제가 있음을 확인할 수 있다.



그러나 CPPCheck는 이 중 버퍼 오버플로우 가능성은 체크를 못하고 3곳의 오류만을 찾아냈다.

다음 코드를 검사해보자.

```
void main(char* str)
{
    char buf[8];
    strcpy(buf, "Hello world!");
    printf("%d\n", *buf);
}
```



[그림 3-9] file2.c 에 대한 오류

Cppcheck가 ‘Out of Bound’ 에러가 났음을 보여준다.

(5) 자동 검사 툴의 한계를 확인한다.

위 두 사례에서 보는 바와 같이 같은 에러 사항 ‘버퍼 오버플로우’에 대해서 경우에 따라 에러를 검출해낼 수도 있고 아닐 수도 있다. 따라서 자동 검사 툴로 검사를 해서 오류가 없다고 해서 무결점의 코드라고 할 수 없음을 검사자는 인식하여야 한다.

③ RoboCon 프로그램을 정적 분석한다.

학습모듈 1, 2에서 작성 및 검사를 수행했던 RoboCon 프로그램에 대해 정적 분석한다.

1. Cppcheck 도구를 열고 설정을 한다.

검사 메뉴를 눌러 하위 메뉴에서 운영체제 환경과 작성한 프로그램의 언어를 선택한다.

2. 프로그램을 검사한다.

‘검사-파일’을 선택해서 각각의 소스 파일을 검사해도 되고 ‘검사-디렉토리’를 선택해서 한꺼번에 프로그램을 구성하는 모든 파일을 검사해도 된다.

3. 에러를 기록하고 수정한다.

검사 결과 에러가 검출되면 우선 그 사항을 기록한다. 결함 상태는 인지, 해결, 무시로 나눌 수 있다.

- 인지: 결함의 내용과 위치를 확인한 상태
- 해결: 코드를 수정하여 결함을 해결한 상태
- 무시: 도구는 결함이라고 검출했으나 프로그래머의 의도 하에 만들어진 코드로 수정이 필요 없는 상태

<표 3-9> 에러 기록표 사례

순번	파일명	결함 위치	결함 유형 및 내용	결함 상태
1	file1.c	8	Memory leak: buf	인지
2	file1.c	13	Mismatching allocation and deallocation:buf	인지
3	file1.c	14	Resource leak: file	인지
:	:	:	:	

이후 에러를 확인하고 수정한다. 에러가 나오는 줄을 더블클릭하면 소스가 열려 편집 가능하다. 에러 수정 후에는 반드시 컴파일러(DEV-C++)에 넣어 컴파일 및 실행을 한 후 다시 검사한다. 최종적으로 에러가 없어질 때까지 이 과정을 반복한다. 결함 상태 모두 해결 혹은 무시로 되어 있어야 한다.

4. 코드리뷰를 수행한다.

도구로 검사할 수 없는 부분이나 프로그램 로직 상의 오류는 검사자가 코드와 설계 자료를 보면서 검출해 내야 한다. 이 부분까지 마쳐야 정적 분석을 완료했다고 할 수 있다.

④ 정적 분석 결과보고서를 작성한다.

정적 분석 완료 후 정적 분석 결과보고서를 아래와 같이 작성한다.

<표 3-10> 정적 분석 결과보고서 예시

XX 소프트웨어 정적 분석 결과보고서

1. 요약

정적 분석 구분	사용 도구/방법	결함 개수	조치율
코딩 규칙 검증	코드리뷰(동료검토)	2	100%
실행 시간 오류	CPPCheck	10	100%

2. 상세

순번	파일명	결함 위치	결함 유형 및 내용	결함 상태
1	file1.c	8	Memory leak: buf	해결
2	file1.c	13	Mismatching allocation and deallocation:buf	해결
3	file1.c	14	Resource leak: file	해결
:	:	:	:	

3. 기타

수행 내용 3 / 동적 분석 수행하기

재료 · 자료

- 노트 및 필기도구
- 소프트웨어 요구 사항 명세서
- 기타 소프트웨어 설계 산출물 (소프트웨어 설계 명세서, 클래스 다이어그램 등)
- 시험 대상 프로그램(RoboCon) 소스 코드
- 정적 분석 시험 절차서

기기(장비 · 공구)

- 컴퓨터, 윈도우즈 XP 이상
- 사무용 프로그램 (아래한글 or MS Office)
- 프린터
- 인터넷

안전 · 유의 사항

- 공개 소프트웨어를 다운로드하여 사용할 경우 라이선스를 반드시 확인하여 사용 여부를 결정하여야 한다. 일부 소프트웨어는 공개했다가 셰어웨어나 유료 판매 정책으로 수정할 수도 있다.
- 학습을 위해 다운로드한 프로그램은 학습이 완료된 후 학습장의 지침을 확인하여 필요 시 프로그램을 제거해야 한다.

수행 순서

① 동적 시험을 준비한다.

동적 시험은 코드를 실행하면서 동작의 오류를 검사하는 행위이므로 코드를 실행하기 위한 환경을 갖추어야 한다. 만일, 자동화 도구가 있다면 자동화 도구를 실행시키고 시험을 위한 세팅을 해야 한다.

1. 시험 절차서를 검토한다.

동적 시험을 수행하기 위해 제일 먼저 해야 할 일은 시험 절차서를 검토하고 시험 환경을 갖추는 것이다. 시험 절차서에는 시험에 필요한 하드웨어, 소프트웨어, 시험에 필요한 지원 장비 등이 명시되어 있으니 이와 함께 기록을 위한 도구 등을 준비한다.

2. DEV-C++ 프로그램을 실행한다.

RoboCon 프로그램 소스를 읽어들이기 위해 프로그램 개발 시 사용하였던 DEV-C++를 실행하고 RoboCon 프로그램 소스를 불러온다.

② 동적 시험을 실시한다.

1. 시험 수행할 테스트 케이스를 검토한다.

시험 절차서의 순서대로 첫 번째 테스트 케이스를 열어 테스트를 수행할 모듈과 입출력 값, 절차 등을 확인한다.

2. 필요한 경우 테스트 드라이버 혹은 스텝을 작성한다.

각 모듈에 대한 테스트를 하기 위해서는 그 모듈을 불러들이기 위한 테스트 드라이버나 혹은 그 모듈이 호출하는 다른 모듈을 대신해주는 스텝을 작성해야 한다.

3. 테스트 케이스를 수행하고 결과를 기록한다.

1개의 테스트 케이스 내에 있는 절차대로 테스트 케이스를 수행하고 그 결과값과 합부 판단을 아래와 같이 기록한다.

<표 3-11> 테스트 케이스 예제

프로그램 명 : RoboCon		단계 : 단위 시험			
테스트 케이스					
Test Case ID : TC-1-XX		Req ID : N/A			
Test Case 설명 : 함수명 BIT_Out(Res) A 입력값과 B 입력값을 확인하여 BIT 테스트 결과를 내주는 모듈					
테스트 조건 : 상위 모듈 000 모듈의 테스트 드라이버를 작성해야 한다.					
단계	입력값	예상 출력값	실행 결과	Pass/Fail	조치 사항
1	A=1, B=1	Res=1	1	Pass	없음
2	A=0, B=1	Res=0	0	Pass	없음

이러한 절차대로 각 테스트 케이스를 시험 후 결과를 기록한다. 모든 조건문에 대해 True와 False의 경로를 다 거쳤는지 확인하여야 한다.

③ 결과보고서를 작성한다.

동적 분석 완료 후 결과보고서를 아래와 같이 작성한다.

<표 3-12> 동적 분석 결과보고서 예시

XX 소프트웨어 동적 분석 결과보고서

1. 요약

순번	분류	결과	비고
1	문장 커버리지	100%	N/A
2	분기 커버리지	95%	Unreachable Code 있음.

2. 상세

순번	클래스명	함수명	테스트 케이스 수	성공수	문장 커버리지
1	AAA	A_aa	4	4	100%
2	AAA	A_bb	2	2	100%
3	BBB	B_aa	3	3	100%
:	:	:	:		

3. 기타

수행 tip

- 학습 내용 1 단위기능 시험평가하기의 테스트 케이스를 참조로 작성한다.

교수 방법

- 소프트웨어의 신뢰성 시험이 중요하다는 점을 ‘아리안 5호 폭발’ 과 같은 과거 사고 사례를 통해 강조하여 학습자들에게 인식되게 한다.
- 중요한 코딩 규칙에 대해서는 잘못된 예시 코드를 만들어 설명하고 학습자들이 이를 수정하면서 훈련하도록 한다.
- 자동화 도구가 검출해내지 못하는 논리적인 오류들에 대해 설명하고 이의 예시 코드를 만들어서 설명하도록 한다.
- 요구 사항 관리, 설계 및 모델링, 구현, 테스트, 형상 관리, 품질관리, 이슈 관리, 프로젝트 관리 등 소프트웨어 개발 관련 다양한 자동화 도구에 대해 설명한다.
- 정적 분석이나 동적 분석 시 발생하는 현상과 이슈 등에 대해 정확히 기록하도록 유도한다.

학습 방법

- 과거 사고 사례를 통해 소프트웨어의 신뢰성 시험이 중요하다는 점을 인식한다.
- 중요한 코딩 규칙에 대해서는 잘못된 예시 코드를 수정하는 훈련을 통해 내재화, 습관화할 수 있도록 하여야 한다.
- 자동화 도구가 모든 논리 오류를 검출해내지 못하는 점을 인식하고 예시 코드를 통해 이를 이해하도록 한다.
- 자동화 도구에 대해서 사전에 공부를 한다. 소프트웨어 개발 관련 다양한 분야의 자동화 도구들에 대해 용도와 종류 등에 대해 소프트웨어 공학센터 홈페이지(<http://www.sw-eng.kr/member/000000000000000032236/Cms/BoardView.do>) 를 통해 살펴본다.
- 정적 분석이나 동적 분석 시 발생하는 현상과 이슈 등에 대해 정확히 기록하는 습관을 갖도록 한다.

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	평가 항목	성취수준		
		상	중	하
신뢰성 시험평가	- 로봇 소프트웨어 신뢰성 시험평가를 위한 시험절차서를 작성할 수 있다.			
	- 시험절차서에 따라 신뢰성 시험평가 항목에 대한 세부 계획 및 적합한 자동화 시험도구를 채택할 수 있다.			
	- 신뢰성 시험평가 항목에 대한 세부계획 및 적합한 자동화 시험도구를 채택할 수 있다.			
	- 자동화 시험도구를 이용하여 시험 평가항목에 대한 신뢰성 시험을 실시할 수 있다.			
	- 결과보고서는 산출물 문서에 포함하여 추적성을 유지하도록 작성할 수 있다.			

평가 방법

- 포트폴리오

학습 내용	평가 항목	성취수준		
		상	중	하
신뢰성 시험평가	- 신뢰성 시험 절차서			
	- 테스트 케이스			
	- 자동화 도구(CPPCheck) 사용 스킬			
	- 테스트 드라이버 및 스텝			
	- 정적 분석 결과보고서			
	- 동적 분석 결과보고서			

• 문제 해결 시나리오

학습 내용	평가 항목	성취수준		
		상	중	하
신뢰성 시험평가	- 설계 문서를 보고 테스트 케이스 작성			
	- 코드 커버리지 목표에 맞는 테스트 케이스 횟수 정의			
	- 정적 분석 시 검출된 오류 수정			
	- 동적 분석 시 검출된 오류 수정			
	- 결과보고서 작성			

• 서술형 시험

학습 내용	평가 항목	성취수준		
		상	중	하
신뢰성 시험평가	- 정적 분석과 동적 분석의 이해			
	- 코드 커버리지의 종류			
	- 문장 커버리지와 분기 커버리지			
	- 조건문에 대해 문장 커버리지 / 분기 커버리지 만족하기 위한 테스트 케이스 횟수 도출			
	- 정적 분석 오류 항목에 대한 설명			
	- 정적 분석 오류 항목 수정하기			

• 평가자 질문

학습 내용	평가 항목	성취수준		
		상	중	하
신뢰성 시험평가	- 정적 분석과 동적 분석의 이해			
	- 코드 커버리지의 종류			
	- 문장 커버리지와 분기 커버리지			
	- 조건문에 대해 문장 커버리지 / 분기 커버리지 만족하기 위한 테스트 케이스 횟수 도출			
	- 정적 분석 오류 항목에 대한 설명			
	- 정적 분석 오류 항목 수정하기			

• 일지/저널

학습 내용	평가 항목	성취수준		
		상	중	하
신뢰성 시험평가	- 테스트 케이스 작성 시 커버리지 목표 도달 여부			
	- 시험 시 과정 및 결과 특이사항 등 기록			
	- 정적 분석 오류 시 처리 과정			
	- 동적 분석 오류 시 처리 과정			

• 작업장 평가

학습 내용	평가 항목	성취수준		
		상	중	하
신뢰성 시험평가	- 시험 시 과정 및 결과 특이사항 등 기록			
	- 자동화 도구 사용 숙련 여부			
	- 정적 분석 오류 시 처리 과정			
	- 동적 분석 오류 시 처리 과정			

피드백

1. 포트폴리오
 - 작성된 테스트 케이스와 테스트 드라이버 및 스텝에 대해 검토하고 다른 방법 등에 대해 설명 후 다르게 작성해 보라고 한다.
2. 문제 해결 시나리오
 - 코드 커버리지의 목표값을 수정해서 제시하고 그에 따른 테스트 케이스를 정의해보라고 한다.
3. 서술형 시험
 - 코드 커버리지의 목표값을 수정해서 제시하고 그에 따른 테스트 케이스를 정의해보라고 한다.
4. 평가자 질문
 - 정적 분석과 동적 분석에 대해 다양한 산업 분야에서 어떻게 적용되는지 조사하여 발표하도록 한다.
5. 일지/저널
 - 각각의 테스트 케이스 수행 후 커버리지 결과값이 어느 정도 되었는지 제출을 요청하고 목표값 대비 현재 상태를 확인하도록 한다.
6. 작업장 평가
 - 시험 결과 오류가 나올 경우 각각의 오류 처리 보고서를 제출하도록 하고 검토 후 의견을 덧붙여 회신한다.



- 김모세(2015). 『코드품질 시각화의 정석』. 지앤선.
- 김종훈, 김종진(2013). 『프로그래밍 언어론: 쉽게 배우는 언어의 원리와 구조』. 한빛아카데미.
- 조대협(2015). 『소프트웨어 개발과 테스트』. 프리렉.
- 한혁수(2008). 『소프트웨어 공학의 소개』. 홍릉과학출판사.
- 도로시 그레이엄·마트 퓨스터([2012] 2014). 『소프트웨어 테스트 자동화 (Experiences of test automation)』. 여용구·김윤명·황영석·성찬혁(역). 에이콘출판.
- 케네스 퍼그([2011] 2014). 『린 애자일 기법을 활용한 테스트 주도 개발(lean-agile acceptance test-driven development)』. 이주형, 제갈호준(역). 에이콘출판.
- 오오츠카 히로키([2015] 2015). 『소셜 코딩으로 이끄는 GitHub 실천기술』. 윤인성(역). 제이펍.
- IEEE. IEEE STD 829. Institute of Electrical and Electronics Engineers



작업 포트폴리오

서식 1. 테스트 케이스

테스트 케이스

프로그램 명 :			단계 : 통합 시험		
Test Case ID :			Req ID :		
시험자 :			시험일 :		
Test Case 설명 :					
테스트 조건 :					
단계	입력값	예상출력값	실행결과	Pass/Fail	조치사항
참고 사항 :					

NCS학습모듈 개발이력

발행일	2016년 12월 31일		
세분류명	로봇소프트웨어개발(19030803)		
개발기관	부경대학교 산학협력단, 한국직업능력개발원		
집필진	이경창(부경대학교)*	검토진	박영제(성균관대학교)
	주문갑(부경대학교)		김종형(서울과학기술대학교)
	김현희(부경대학교)		노재상(에이오비)
	진태석(동서대학교)		전상원(파인로보틱스)
	김종욱(동아대학교)		이래찬(대광테크)
	김만호(동의과학대학교)		
	하경남(한국생산기술연구원)		
	정현권(유콘시스템)		
발행일	2018년 12월 31일		
학습모듈명	로봇 소프트웨어 시험평가(LM1903080310_14v1)		
개발기관	한국직업능력개발원		

로봇 소프트웨어 시험평가(LM1903080310_14v1)

저작권자	교육부
연구기관	한국직업능력개발원
발행일	2018. 12. 31.

※ 이 학습모듈은 자격기본법 시행령(제8조 국가직무능력표준의 활용)에 의거하여 개발 하였으며, NCS통합포털사이트(<http://www.ncs.go.kr>)에서 다운로드 할 수 있습니다.



www.ncs.go.kr